



Universidade do Minho
Licenciatura em Engenharia Informática

Redes de Computadores

Trabalho Prático - Desenvolvimento de um Packet Sniffer e Analisador de Tráfego

Ano Letivo de 2025/2026

Gonçalo Fernandes da Silva e Sousa - **A110977**

Pedro Tiago Moniz Morais - **A109719**

Tiago da Cunha Pereira - **A112010**

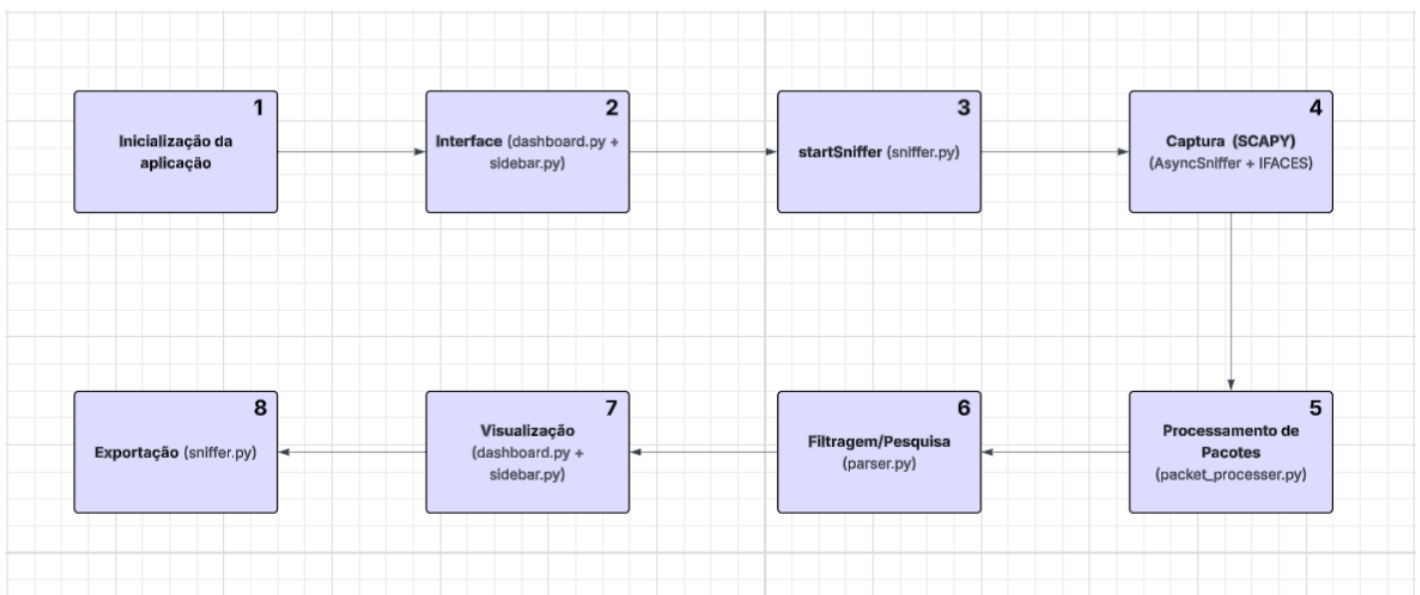
1. Arquitetura Implementada

O sistema desenvolvido segue uma arquitetura modular, dividindo-se em quatro componentes principais: captura de pacotes, processamento de protocolos, filtragem de tráfego e a interface gráfica.

A captura de tráfego é realizada através da biblioteca **Scapy**, segundo as indicações referidas no enunciado do trabalho prático, que permite a recolha de pacotes em tempo real a partir de uma interface de rede selecionada pelo utilizador. Cada pacote capturado é posteriormente encaminhado para um módulo de processamento responsável pela identificação e extração de informação relevante dos diferentes protocolos de rede, incluindo Ethernet, ARP, IP, ICMP, TCP e UDP.

Após o processamento, os pacotes são armazenados num histórico interno, permitindo a sua análise posterior. Sobre este histórico é aplicado um sistema de filtragem que interpreta os filtros definidos pelo utilizador.

Por fim, os dados são apresentados através de uma interface gráfica interativa, desenvolvida em **CustomTkinter**, que permite visualizar, navegar e analisar os pacotes capturados em tempo real. A aplicação inclui ainda um mecanismo de exportação de dados, permitindo guardar o histórico de pacotes em ficheiro com formato **json** para análise posterior. De seguida apresenta-se a arquitetura referente ao nosso Sniffer de forma detalhada, onde em cada quadrado são apresentados todos os passos juntamente com o ficheiro com a determinada funcionalidade apresentada.



1- Inicialização da Aplicação

A inicialização da aplicação é responsável por configurar o ambiente gráfico, preparar os componentes da interface e garantir que o utilizador selecione uma interface de

rede disponível no sistema, sendo esta seleção obrigatória para que este consiga visualizar a captura de tráfego na determinada interface escolhida.

Após a configuração inicial da interface e seleção da interface de rede, a aplicação entra em modo de execução contínua, ficando pronta para iniciar a captura. Este primeiro ponto garante que todos os componentes do sistema são corretamente inicializados e que o utilizador dispõe das condições necessárias para iniciar a captura e análise de tráfego de rede.

2- Interface

2.1- Escolha da Interface

Na fase inicial da execução da aplicação, o utilizador é confrontado com um processo de seleção da interface de rede através da qual será efetuada a captura de tráfego, ou seja, o utilizador tem que escolher de forma obrigatória a interface que pretende para capturar o tráfego.

Esta funcionalidade encontra-se implementada no módulo **dashboard.py**, através da função **createInterfaceSelector**. Esta função abre uma janela de seleção (**CTkToplevel**) que apresenta todas as interfaces de rede disponíveis no sistema, obtidas através da função **getInterfaces()** do módulo **sniffer.py**.

Cada interface apresentada permite ao utilizador visualizar o seu nome e descrição, sendo disponibilizado um botão de seleção associado a cada uma delas. Ao selecionar uma interface, é chamada a função **setInterface()**, responsável por definir a interface ativa para captura de pacotes.

Desta forma, esta etapa inicial garante que o sistema de captura está corretamente associado à interface de rede pretendida pelo utilizador.

2.2- Interface principal (Dashboard)

Após a seleção da interface de rede, a aplicação avança para o ecrã principal, designado por **dashboard**, onde é apresentada toda a informação relativa ao tráfego de rede capturado em tempo real.

Esta componente representa a principal área de interação do utilizador com o sistema. O ecrã principal encontra-se implementado no módulo **dashboard.py**, sendo construído através da função **createDashboard**. Esta função é responsável pela criação de todos os elementos visuais da interface, incluindo a barra de filtragem e pesquisa, área de listagem dos pacotes e as partes de navegação.

A apresentação dos pacotes capturados é realizada através de uma lista dinâmica de componentes visuais os **accordions**, onde cada pacote é representado individualmente com a respetiva informação relevante, o protocolo utilizado, endereço de origem, endereço de destino e timestamp de captura.

A atualização da interface ocorre através da função **receivePacket**, que recebe os dados processados pelo sistema de captura e os encaminha para a função **editAccordion**, responsável por preencher os campos visuais de cada pacote na interface.

Além da visualização dos accordions referentes aos pacotes, o ecrã principal também disponibiliza funcionalidades de interação com o utilizador, nomeadamente:

- Barra de pesquisa para filtragem de pacotes (**setFilter**)
- Navegação entre páginas de resultados (**setPage**, **setNextPage**, **setPreviousPage**)
- Contador de pacotes capturados (**setPacketCounter**)
- Controlo de paginação (**setMaxPage**)

Desta forma, o dashboard não se limita à apresentação de dados, mas também permite a sua organização proporcionando uma visualização estruturada e interativa do tráfego de rede capturado a cada utilizador.

3- startSniffer()

A função **startSniffer()** presente no ficheiro **sniffer.py** é responsável por iniciar o motor de captura de pacotes da aplicação, dando início ao processo de monitorização de tráfego de rede em tempo real. Quando esta função é executada, o sistema verifica primeiro se o sniffer já foi criado; caso ainda não exista, é chamado o método **setSniffer()** e o prepara para capturar pacotes na interface de rede previamente selecionada pelo utilizador.

Antes de iniciar uma nova sessão de captura, o sistema limpa o histórico de pacotes armazenados através de **_history.clear()**, garantindo que não existem dados anteriores a interferir com a nova análise. De seguida, a captura é iniciada com **_sniffer.start()**, colocando o sistema em modo ativo de observação contínua, onde cada pacote recebido será automaticamente processado pela função (**_processPacket**).

Em paralelo com a captura, a função também trata da sincronização com a interface gráfica. A variável de paginação é reiniciada para a primeira página, os botões de navegação são ocultados e o estado visual da aplicação é atualizado através de **setStartStatus()**, indicando ao utilizador que a captura foi iniciada com sucesso, ou foi parada com sucesso através da **setStopStatus()**.

Desta forma, o **startSniffer()** é a função que inicia a captura de pacotes e liga o sistema de recolha de dados de rede à interface gráfica, permitindo que os pacotes capturados sejam processados e apresentados ao utilizador em tempo real.

4- Captura

A fase de captura de pacotes é realizada pelo **AsyncSniffer** da biblioteca **Scapy**, que é configurado na função **setSniffer()**, sendo especificada a interface de captura e a operação para processar cada pacote recebido, e executado através de **startSniffer()**. Depois de iniciado, o sniffer passa a escutar continuamente a interface de rede selecionada e, sempre que um pacote é detetado, este é automaticamente enviado para a função **_processPacket()** definida no módulo **sniffer.py**.

Assim, a fase de captura corresponde ao momento em que os pacotes capturados pelo scapy são encaminhados para a função de processamento passada no momento da criação do sniffer, **_processPacket()**.

5- Processamento de Pacotes

A fase de processamento de pacotes ocorre imediatamente após a captura, sendo centralizada na função **_processPacket()** presente no ficheiro **sniffer.py**. Esta função recebe cada pacote capturado pelo **AsyncSniffer** e transforma-o numa estrutura de dados organizada, preparada para análise e apresentação na interface gráfica, como vimos no ponto anterior da Captura.

Inicialmente, o pacote é analisado por diferentes processadores definidos no ficheiro **packet_processor.py**, onde cada camada do modelo de rede é tratada separadamente. As funções **EthernetProcessor**, **ARPProcesser**, **IPProcessor**, **ICMPProcesser**, **TCPProcessor** e **UDPProcesser** verificam se o pacote contém os respetivos protocolos e, caso isso se verifique, extraem os campos relevantes de cada um. Esta abordagem permite decompor o pacote em várias camadas lógicas, facilitando a interpretação do tráfego de rede.

Após este processamento, os dados de cada camada são organizados em estruturas como **layer2**, **layer3** e **layer4**, representando as camadas da pilha protocolar. Em simultâneo, é determinado o protocolo do pacote.

Por fim, é aplicada a lógica de apresentação dos pacotes na interface. O tamanho do histórico filtrado é recalculado e, com base nesse valor, determina-se o número total de páginas disponíveis, dividindo o total de pacotes pelo número de pacotes exibidos por página (10).

De seguida, é verificado se o novo pacote pertence à página atualmente visível. Caso isso se confirme, o pacote é imediatamente apresentado na interface através da função **receivePacket**. Caso contrário, não é mostrado nessa página e é ativado o botão “Próximo”, indicando ao utilizador que existem mais páginas disponíveis para consulta.

Por último, é atualizado o contador global de pacotes, garantindo que a interface reflete corretamente o número total de pacotes capturados, independentemente da filtragem aplicada.

6- Filtragem/ Pesquisa

A fase de filtragem e pesquisa é responsável por permitir ao utilizador restringir os pacotes apresentados com base em determinados critérios, como por exemplo o tipo de pacote. Esta funcionalidade está implementada principalmente no ficheiro **parser.py** e é integrada no fluxo da aplicação através da função **parse_filter()**, sendo utilizada no **sniffer.py** sempre que o utilizador introduz um filtro na interface gráfica.

Quando o utilizador escreve um filtro na barra de pesquisa (por exemplo **protocol==ICMP**), esse valor é capturado pela interface e são apresentados todos os pacotes restritos a esse determinado protocolo, sendo que nessa fase de captura restrita são capturados os pacotes que já tínhamos e os que continuaram a vir dentro do filtro imposto. Uma característica

importante do nosso Sniffer, semelhante ao Wireshark, é que a aplicação do filtro não interfere no processo de captura de tráfego de rede. Ou seja, mesmo quando um filtro é definido pelo utilizador, o sistema continua a capturar todos os pacotes que passam pela interface de rede selecionada.

A função **parse_filter()** é responsável por interpretar a expressão textual do filtro. Esta função analisa a string recebida bem como os seus operadores de modo a satisfazerem as condições dos utilizadores. Ainda assim, para cada pacote, a função **_getDataFilter()** verifica se o campo indicado no filtro existe e compara o seu valor com o critério definido. Se a condição for verdadeira, o pacote é incluído no conjunto de resultados filtrados. Caso contrário, é ignorado. Este processo permite filtrar tanto por campos simples como protocolo ou endereço IP, como por outro tipo de condições.

No sniffer.py, este mecanismo é utilizado dentro da função **_processPacket()**, onde cada pacote capturado é testado contra o filtro ativo. Apenas os pacotes que cumprem as condições definidas pelo utilizador são adicionados ao conjunto de resultados visíveis na interface. Assim, o utilizador consegue ver os pacotes que decidiu filtrar em tempo real sem interferir com a captura contínua do tráfego.

7- Visualização

A fase de visualização corresponde à forma como os pacotes capturados e processados são apresentados ao utilizador através da interface, sendo implementada principalmente no ficheiro **dashboard.py**. Esta camada tem como objetivo transformar os dados estruturados gerados pelo sniffer num formato compreensível para o utilizador, permitindo a análise do tráfego de rede em tempo real.

A interface utiliza um sistema de “**accordions**”, onde cada pacote é representado por um bloco. Na vista simplificada, o utilizador consegue rapidamente identificar o tipo de tráfego e os principais campos de cada pacote. Ao expandir um pacote, é apresentada informação mais detalhada, organizada por camadas (Layer 2, Layer 3 e Layer 4), permitindo uma análise mais detalhada sobre cada pacote.

Desta forma, a camada de visualização é responsável por converter dados técnicos de rede numa representação gráfica interativa, facilitando a interpretação e análise do tráfego capturado, de forma mais detalhada caso a seleção do determinado pacote.

8- Exportação

A fase de exportação permite ao utilizador guardar o histórico dos pacotes capturados para análise posterior. Aqui há duas formas de guardar de forma persistente. Sempre que é iniciada uma captura, os pacotes que vão sendo capturados vão sendo salvos de forma persistente num ficheiro criado no momento do início da captura. Para além disto, na nossa sidebar temos um botão que permite ao utilizador fazer download do tráfego (filtrado ou não) e guardá-lo no seu sistema de ficheiros, indicando o nome do ficheiro.

2. Catalogação dos protocolos identificados e a sua análise

O nosso sniffer identifica os protocolos através da análise dos cabeçalhos de cada camada da pilha protocolar, recorrendo a funções específicas para cada protocolo. A identificação é feita de forma hierárquica, desde a camada 2 até à camada 4 (com eventual deteção de portas referentes a protocolos camada aplicacional), permitindo uma classificação correta do tráfego capturado. Todo o código apresentado abaixo, necessário para uma melhor compreensão de como cada pacote é processado, encontra-se no ficheiro **packet_processor.py**. Cada protocolo é identificado num pacote se, aplicando a função **haslayer(<Protocolo>)** sobre o pacote capturado, é retornado um valor **true**.

Para cada pacote capturado, é apresentado inicialmente um resumo com as informações mais relevantes, nomeadamente:

- *Packet id*: identificador único do pacote capturado;
- *Protocolo*: protocolo da camada mais alta identificado no pacote;
- *Origem e destino*: endereços IP de origem e destino, ou endereços MAC caso o pacote não possua encapsulamento de camada 3;
- *Tamanho*: tamanho total do pacote em bytes, incluindo o cabeçalho da camada 2;
- *Info*: informação adicional sobre o pacote, dependendo do protocolo identificado;
- *Timestamp*: instante em que o pacote foi capturado.

Como por exemplo:

>	1 HTTPS	Fonte: 104.18.39.21	Destino: 172.26.229.52	Tamanho: 78	Info: TCP (HTTPS) 443 → 57374	30/04/2026 14:56:42
>	2 HTTPS	Fonte: 172.26.229.52	Destino: 104.18.39.21	Tamanho: 82	Info: TCP (HTTPS) 57374 → 443	30/04/2026 14:56:42
>	3 HTTPS	Fonte: 104.18.39.21	Destino: 172.26.229.52	Tamanho: 54	Info: TCP ACK (HTTPS) 443 → 57374	30/04/2026 14:56:42

Se o utilizador quiser obter mais informações, poderá abrir a tab do pacote capturado e ver informações mais específicas daquele pacote, organizadas por layer, por exemplo, para um pacote HTTPS:

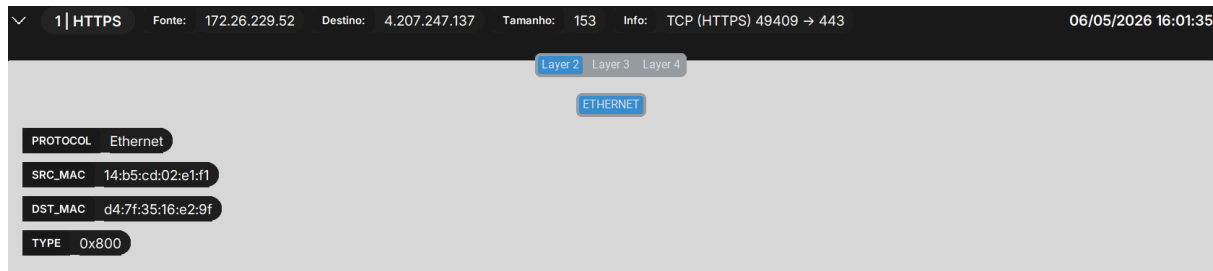
Cabeçalho Layer 4:

1 HTTPS	Fonte: 172.26.229.52	Destino: 4.207.247.137	Tamanho: 153	Info: TCP (HTTPS) 49409 → 443	06/05/2026 16:01:35
Layer 2 Layer 3 Layer 4					
PROTOCOL TCP					
SRCPORT 49409					
DSTPORT 443					
APP_PROTOCOL HTTPS					
SUMMARY TCP (HTTPS) 49409 → 443					

Cabeçalho Layer 3:

1 HTTPS	Fonte: 172.26.229.52	Destino: 4.207.247.137	Tamanho: 153	Info: TCP (HTTPS) 49409 → 443	06/05/2026 16:01:35
Layer 2 Layer 3 Layer 4					
IP					
VERSION 4					
HEADER_LENGTH 5					
LEN 139					
ID 57555					
FLAGS DF					
OFFSET 0					
PROTOCOL 6					
TTL 128					
SRC 172.26.229.52					
DST 4.207.247.137					
SUMMARY Datagrama Normal					

Cabeçalho Layer 2:



Layer 2

Ethernet

O protocolo Ethernet serve para permitir a comunicação entre dispositivos numa rede local, identificando-os através dos endereços MAC e transportando os dados encapsulados até ao próximo destino. Para pacotes que tenham levado encapsulamento deste protocolo, especificamos os campos correspondentes aos endereços MAC de origem e destino e o campo Type que identifica de que tipo, ou seja qual o protocolo que originou o payload que vai encapsulado na trama Ethernet.. Estes elementos permitem identificar os dispositivos na rede local e o tipo de protocolo encapsulado, como IPv4 ou ARP, garantindo o correto encaminhamento dos pacotes até ao próximo salto.

```
def EthernetProcessor(packet: Packet):  
    if packet.haslayer(Ether):  
        eth = packet[Ether]  
  
        return {  
            "protocol": "Ethernet",  
            "src_mac": eth.src,  
            "dst_mac": eth.dst,  
            "type": hex(eth.type),  
        }  
    return None
```

ARP

O protocolo ARP é utilizado para resolver endereços IP em endereços MAC dentro de uma rede local. De forma geral, um dispositivo envia um pedido (ARP Request) para descobrir o endereço MAC associado a um determinado IP, e o dispositivo correspondente responde com um ARP Reply indicando o seu MAC.

No código, são analisados alguns campos principais: o campo *op*, que indica o tipo de mensagem (1 para request e 2 para reply), os endereços do sender MAC (*hwsrc*) e target MAC (*hwdst*), e os endereços IP do emissor (*psrc*) e do alvo (*pdst*).

Com base no valor de *op*, são definidos os campos *summary* e *info*:

- Se for um ARP Request (*op* = 1), o *summary* é “ARP request” e o *info* indica o pedido no formato “Quem é pdst? Digam a psrc”.
- Se for um ARP Reply (*op* = 2), o *summary* é “ARP reply” e o *info* indica a associação no formato “psrc está em hwsrc”.
- Para outros valores, o *summary* assume “ARP op X” e o *info* é uma descrição genérica do protocolo.

```
def ARPProcesser(packet: Packet):
    if packet.haslayer(ARP):
        arp = packet[ARP]

        if arp.op == 1:
            summary = "ARP request"
            info = f"Quem é {arp.pdst}? Digam a {arp.psrc}"
        elif arp.op == 2:
            summary = "ARP reply"
            info = f"{arp.psrc} está em {arp.hwsrc}"
        else:
            summary = f"ARP op {arp.op}"
            info = "Protocolo de controlo ARP"

        return {
            "protocol": "ARP",
            "sender_mac": arp.hwsrc,
            "target_mac": arp.hwdst,
            "info": info,
            "summary": summary
        }
    return None
```

Layer 3

IPv4

O protocolo IP é responsável pelo encaminhamento de pacotes entre diferentes redes IP, utilizando os endereços IP de origem e destino. Cada pacote contém um cabeçalho IPv4 com a informação necessária para esse processo.

No código, são analisados vários campos do cabeçalho IP, nomeadamente a versão (*version*), o tamanho do cabeçalho (*ihl*), o tamanho total do pacote (*len*), o identificador (*id*), os *flags*, o *offset* de fragmentação, o campo *protocol*, o *TTL*, e os endereços IP de origem (*src*) e destino (*dst*).

O campo *protocol* permite identificar qual o protocolo encapsulado no payload, sendo convertido para uma forma legível através de um mapeamento interno, a partir do inteiro que identifica protocolo, sendo possível identificar depois no campo *protocol_name*, com base no campo *protocol*, o nome do protocolo identificado em específico: ICMP, TCP, UDP, IPv6, OSPF.

Os campos *flags* e *offset* são utilizados para verificar se o pacote foi fragmentado. Com base nisso, é construído o campo *summary*: se o pacote estiver fragmentado (ou seja, *frag*

≠ 0 ou o flag *MF* estiver ativo), o *summary* é definido como “Datagrama Fragmentado”; caso contrário, assume o valor “Datagrama Normal”.

```
def IPProcessor(packet: Packet):
    if packet.haslayer(IP):
        ip = packet[IP]

        proto_map = {
            1: "ICMP",
            6: "TCP",
            17: "UDP",
            41: "IPv6",
            89: "OSPF"
        }

        proto_name = proto_map.get(ip.proto, str(ip.proto))

        return {
            "version": ip.version,
            "header_length": ip.ihl,
            "len": ip.len,
            "id": ip.id,
            "flags": str(ip.flags) if ip.flags else "0",
            "offset": ip.frag,
            "protocol": ip.proto,
            "protocol_name": proto_name,
            "ttl": ip.ttl,
            "src": ip.src,
            "dst": ip.dst,
            "summary": "Datagrama Fragmentado" if (ip.frag != 0 or ip.flags.MF) else "Datagrama Normal"
        }
    return None
```

IPv6

O protocolo IPv6 é a versão mais recente do protocolo IP e é utilizado para encaminhar pacotes entre diferentes redes, usando endereços IPv6 de origem e destino. No código, quando é detectado um pacote IPv6, são analisados alguns campos principais do cabeçalho, como a versão (*version*), a classe de tráfego (*traffic_class*), o flow label (*flow_label*), o tamanho do payload (*payload_length*), o próximo cabeçalho (*next_header*), o hop limit (*hop_limit*) e os endereços IPv6 de origem (*src*) e destino (*dst*).

O campo *next_header* indica qual é o próximo protocolo ou cabeçalho transportado pelo IPv6, por exemplo TCP, UDP ou ICMPv6. O campo *hop_limit* tem uma função semelhante ao TTL no IPv4, limitando o número de saltos que o pacote pode fazer na rede.

Por fim, é definido um *summary* simples com o valor “**Datagrama IPv6**”, indicando que o pacote analisado pertence ao protocolo IPv6.

```
def IPv6Processor(packet: Packet):
    if packet.haslayer(IPv6):
        ip6 = packet[IPv6]

        return {
            "version": ip6.version,
            "traffic_class": ip6.tc,
            "flow_label": ip6.fl,
            "payload_length": ip6.plen,
            "next_header": ip6.nh,
            "hop_limit": ip6.hlim,
            "src": ip6.src,
            "dst": ip6.dst,
            "summary": "Datagrama IPv6"
        }
    return None
```

ICMP

O protocolo ICMP é utilizado para enviar mensagens de controlo e diagnóstico na rede, permitindo verificar a conectividade entre dispositivos e identificar problemas no encaminhamento dos pacotes.

No código, são analisados principalmente os campos *type* e *code* do cabeçalho ICMP. O campo *type* permite identificar o tipo de mensagem, como Echo Request (*type* = 8), Echo Reply (*type* = 0), Destination Unreachable (*type* = 3) e Time Exceeded (*type* = 11), enquanto o campo *code* fornece informação adicional sobre a mensagem.

Com base no valor de *type*, é construído o campo *summary*:

- *type* = 8 → "ICMP echo request"
- *type* = 0 → "ICMP echo reply"
- *type* = 3 → "Destination unreachable"
- *type* = 11 → "Time exceeded"
- outros valores → "ICMP type X"

Além disso, o campo *type* é apresentado juntamente com o seu valor numérico, e o *code* é incluído para fornecer mais detalhe sobre a mensagem, permitindo uma interpretação simples do comportamento do pacote no dashboard.

```
def ICMPProcessor(packet: Packet):  
    if packet.haslayer(ICMP):  
        icmp = packet[ICMP]  
  
        if icmp.type == 8:  
            summary = "ICMP echo request"  
        elif icmp.type == 0:  
            summary = "ICMP echo reply"  
        elif icmp.type == 3:  
            summary = "Destination unreachable"  
        elif icmp.type == 11:  
            summary = "Time exceeded"  
        else:  
            summary = f"ICMP type {icmp.type}"  
  
        return {  
            "protocol": "ICMP",  
            "type": icmp.type,  
            "code": str(icmp.code),  
            "summary": summary,  
        }  
    return None
```

Exemplo de summary de pacotes ICMP

>	7 ICMP	Fonte: 172.26.229.52	Destino: 216.58.198.14	Tamanho: 106	Info: ICMP echo request	30/04/2026 14:58:35
>	8 ICMP	Fonte: 172.26.254.254	Destino: 172.26.229.52	Tamanho: 70	Info: Time exceeded	30/04/2026 14:58:35
>	9 ICMP	Fonte: 172.26.229.52	Destino: 216.58.198.14	Tamanho: 106	Info: ICMP echo request	30/04/2026 14:58:35

Layer 4

O protocolo TCP é utilizado para garantir uma comunicação fiável entre dispositivos, assegurando a entrega correta e ordenada dos dados.

No código, não se incluiu uma grande especificação na análise deste protocolo, uma vez que não foi dado. Ainda são analisados os campos relativos às *flags* e às portas de origem (*sport*) e destino (*dport*). As *flags* permitem identificar o estado da ligação, como SYN (início), SYN-ACK, ACK, FIN (terminação) e RST (reset).

Com base nas *flags*, é construído o campo *summary*:

- “S” → “TCP SYN”
- “SA” → “TCP SYN-ACK”
- “A” → “TCP ACK”
- “F” → “TCP FIN”
- “R” → “TCP RST”
- outros casos → “TCP”

Além disso, as portas são utilizadas para identificar o protocolo de aplicação associado. Se for detetada uma porta conhecida, o *summary* inclui o respetivo protocolo (HTTP, HTTPS, SSH ou FTP). Ainda assim, são apresentadas as portas no formato “porta origem → porta destino”. Desta forma, o código permite não só identificar o tipo de segmento TCP, mas também inferir o serviço associado e apresentar uma descrição simples da comunicação no campo *summary*.

```
def TCPProcessor(packet: Packet):
    if packet.haslayer(TCP) and packet.haslayer(IP):
        tcp = packet[TCP]

        flags = tcp.flags

        if flags == "S":
            summary = "TCP SYN"
        elif flags == "SA":
            summary = "TCP SYN-ACK"
        elif flags == "A":
            summary = "TCP ACK"
        elif flags == "F":
            summary = "TCP FIN"
        elif flags == "R":
            summary = "TCP RST"
        else:
            summary = "TCP"

        # APP LAYER
        app_protocol = None

        if tcp.dport == 80 or tcp.sport == 80:
            app_protocol = "HTTP"
        elif tcp.dport == 443 or tcp.sport == 443:
            app_protocol = "HTTPS"
        elif tcp.dport == 22 or tcp.sport == 22:
            app_protocol = "SSH"
        elif tcp.dport == 21 or tcp.sport == 21:
            app_protocol = "FTP"

        if app_protocol:
            summary += f" ({app_protocol})"

        summary += f" {tcp.sport} → {tcp.dport}"

        return {
            "protocol": "TCP",
            "src_port": tcp.sport,
            "dst_port": tcp.dport,
            "app_protocol": app_protocol,
            "summary": summary,
        }

    return None
```

Exemplo do summary de um pacote TCP (com o protocolo aplicacional HTTPS)

>	1 HTTPS	Fonte: 104.18.39.21	Destino: 172.26.229.52	Tamanho: 78	Info: TCP (HTTPS) 443 → 57374	30/04/2026 14:56:42
>	2 HTTPS	Fonte: 172.26.229.52	Destino: 104.18.39.21	Tamanho: 82	Info: TCP (HTTPS) 57374 → 443	30/04/2026 14:56:42
>	3 HTTPS	Fonte: 104.18.39.21	Destino: 172.26.229.52	Tamanho: 54	Info: TCP ACK (HTTPS) 443 → 57374	30/04/2026 14:56:42

UDP

O protocolo UDP é utilizado para comunicação simples e rápida entre dispositivos, não garantindo a entrega nem a ordem dos pacotes.

No código, são analisadas principalmente as portas de origem (*sport*) e destino (*dport*), que permitem identificar o protocolo de aplicação associado, como DHCP, DNS ou NTP.

Com base nas portas, é definido o protocolo de aplicação (*app_protocol*):

- portas 67 ou 68 → DHCP
- porta 53 → DNS
- porta 123 → NTP

No caso do DHCP, uma vez que foi um protocolo estudado, é ainda analisado o conteúdo da mensagem para identificar o tipo de operação, como Discover, Offer, Request ou ACK, sendo essa informação adicional guardada em *extra*.

O campo *summary* é então construído com base no protocolo identificado. Se existir um protocolo conhecido, o *summary* apresenta o seu nome (por exemplo, “DHCP” ou “DNS”), podendo incluir informação extra no caso do DHCP. Caso contrário, são apresentadas as portas no formato “porta origem → porta destino”.

Desta forma, o código permite identificar rapidamente o tipo de serviço associado ao tráfego UDP e apresentar uma descrição simples da comunicação.

```
def UDPProcessor(packet: Packet):
    if not packet.haslayer(UDP):
        return None

    udp = packet[UDP]

    app_protocol = None
    extra = None

    # DHCP
    if udp.sport in [67, 68] or udp.dport in [67, 68]:
        app_protocol = "DHCP"

        if packet.haslayer(DHCP):
            dhcp = packet[DHCP]

            msg_types = {
                1: "Discover",
                2: "Offer",
                3: "Request",
                5: "ACK",
            }

            for opt in dhcp.options:
                if isinstance(opt, tuple) and opt[0] == "message-type":
                    extra = msg_types.get(opt[1], str(opt[1]))
                    break

    # DNS
    elif udp.dport == 53 or udp.sport == 53:
        app_protocol = "DNS"

    # NTP
    elif udp.dport == 123 or udp.sport == 123:
        app_protocol = "NTP"

    # NTP
    elif udp.dport == 123 or udp.sport == 123:
        app_protocol = "NTP"

    # Summary (igual lógica ao TCP)
    summary = "UDP"

    summary += f" {udp.sport} → {udp.dport}"

    if app_protocol:
        summary += f" {app_protocol}"

    if extra:
        summary += f" {extra}"

    return {
        "protocol": "UDP",
        "srcPrt": udp.sport,
        "dstPrt": udp.dport,
        "app_protocol": app_protocol,
        "summary": summary,
    }
```

Exemplo de um pacote UDP capturado, com identificação do protocolo da camada aplicacional DNS

>	57 DNS	Fonte: 172.26.229.52	Destino: 193.137.16.65	Tamanho: 76	Info: UDP (DNS) 49289 → 53	30/04/2026 15:08:05
>	58 DNS	Fonte: 172.26.229.52	Destino: 193.137.16.65	Tamanho: 76	Info: UDP (DNS) 65363 → 53	30/04/2026 15:08:05
>	63 DNS	Fonte: 193.137.16.65	Destino: 172.26.229.52	Tamanho: 182	Info: UDP (DNS) 53 → 49289	30/04/2026 15:08:05

Lista de filtros e parâmetros de entrada disponíveis

O sistema permite o uso de operadores como `==`, `!=`, `>`, `<`, `>=` e `<=`, bem como operadores lógicos **and** e **or**, possibilitando a construção de expressões definidas pelo utilizador. Os principais parâmetros disponíveis para filtragem correspondem aos campos gerais presentes na estrutura do pacote processado, nomeadamente:

- `protocol` - tipo de protocolo (ex: TCP, UDP, ICMP, ARP, Ethernet, HTTPS, SSH)
- `src` - endereço de origem
- `dst` - endereço de destino
- `packet_id` - identificador do pacote
- `length` - tamanho total do pacote em bytes

que são os campos presentes no summary.

Adicionalmente, como cada pacote pode conter vários níveis protocolares de encapsulamento, o sistema permite aceder aos campos específicos de cada camada através da sintaxe:

- `layerX.campo`

onde X representa a camada protocolar. Assim, é possível filtrar campos pertencentes à camada 2, camada 3 ou camada 4, dependendo da informação disponível no pacote capturado.

Para a camada 2 podem ser filtrados campos de Ethernet ou ARP:

- `layer2.arp.sender_mac`
- `layer2.ethernet.src_mac`
- `layer2.arp.target_mac`
- `layer2.ethernet.dst_mac`
- `layer2.ethernet.type`

Ou então, caso eventualmente o pacote seja apenas encapsulado em layer 2, podemos filtrar diretamente os MACs source e destino através de:

- `src`
- `dst`

Na camada 3, podem ser filtrados campos de IP ou ICMP, como:

- `layer3.ip.src`

- layer3.ip.dst
- layer3.ip.ttl
- layer3.ip.len
- layer3.ip.header_length
- layer3.ip.id
- layer3.ip.protocol
- layer3.ip.flags (MF, DF, 0)
- layer3.ip.offset
- layer3.icmp.type
- layer3.icmp.code
- layer3.ipv6.version
- layer3.ipv6.traffic_class
- layer3.ipv6.flow_label
- layer3.ipv6.payload_length
- layer3.ipv6.next_header
- layer3.ipv6.hop_limit
- layer3.ipv6.src
- layer3.ipv6.dst

Na camada 4, podem ser filtrados campos de TCP ou UDP, como:

- layer4.protocol
- layer4.srcPrt
- layer4.dstPrt
- layer4.app_protocol

Desta forma, para além dos campos gerais do pacote, o utilizador pode criar filtros mais específicos sobre os dados extraídos de cada protocolo. Por exemplo, se o pacote tiver informação IP, é possível filtrar por endereço IP; se tiver informação Ethernet ou ARP, é possível filtrar por endereços MAC.

Alguns exemplos de filtros possíveis são:

- protocol==HTTPS
- src==192.168.1.1
- dst==00:11:22:33:44:55
- protocol==DNS or length>54
- packet_id==50
- protocol==ICMP and length==106
- layer3.ttl>64
- layer4.dstPrt==443
- layer4.app_protocol==DNS
- layer2.ethernet.src_mac==00:11:22:33:44:55

Internamente, o filtro introduzido pelo utilizador é separado em tokens, convertido para uma expressão em notação pós-fixa e depois transformado numa árvore de avaliação. Durante a avaliação, o sistema percorre os pacotes capturados e compara o valor indicado no filtro com o valor real presente na estrutura do pacote. Quando o campo contém pontos, como layer4.dst_port, a função percorre os dicionários internos até encontrar o valor

correspondente. Assim, o sistema permite uma filtragem flexível e dinâmica, adaptada ao nível de detalhe disponível em cada pacote capturado.

Mecanismos de Logging implementados

A aplicação implementa um sistema de registo (logging) que organiza a informação dos pacotes capturados para consulta imediata e análise posterior. Este mecanismo divide-se em três vertentes principais:

Histórico de Pacotes

É o registo de todos os pacotes processados durante a sessão atual são armazenados de forma estruturada. Sempre que um pacote é capturado pelo sniffer, este é enviado para a função **_processPacket**, onde são extraídas as informações relevantes das diferentes camadas de rede. Depois de processado, o pacote é organizado num dicionário **data**, que contém todas as informações. Esse dicionário é depois adicionado à lista global **_history**, permitindo que a aplicação mantenha um histórico temporário de todos os pacotes capturados.

Este histórico funciona em conjunto com a função **parse_filter**, que permite aplicar condições sobre o tráfego capturado. Assim, quando existe um filtro ativo, são apresentados apenas os pacotes que correspondem a esse filtro, caso contrário, é apresentado todo o histórico.

Persistência automática em formato JSON

Durante a captura, a aplicação cria automaticamente um ficheiro de log em formato JSON, com um nome baseado na data e hora de início da sessão, no seguinte formato, **log_YYYYMMDD_HHMMSS.json**. À medida que novos pacotes são processados, estes são acrescentados ao ficheiro através da função **write_to_log**. Nisto, o ficheiro log é processado quando paramos a captura e quando voltarmos a dar start no sniffer irá ser criado outro ficheiro log, ou seja, sempre que paramos é garantido o armazenamento persistente da informação relativa aos pacotes para análise posterior. Esta persistência permite manter um registo permanente dos pacotes capturados, mesmo após o fim da execução da aplicação.

Exportação do histórico manualmente

Durante a captura do tráfego, a aplicação disponibiliza uma forma de persistência manual dos dados através de um botão presente na interface, representado por um ícone de formato quadrado. Este botão permite ao utilizador exportar o histórico de pacotes capturados para um ficheiro JSON no momento em que desejar, possibilitando a recolha da informação selecionada pelo próprio utilizador.

Este mecanismo é considerado manual porque depende diretamente da ação do utilizador, ao contrário do registo automático, que guarda os pacotes continuamente durante a captura. Além disso, a exportação tem em conta os filtros aplicados na interface: caso exista um filtro ativo, são guardados apenas os pacotes que correspondem a esse filtro, caso

contrário, é exportado todo o histórico de pacotes capturados. Desta forma, o utilizador pode guardar apenas a informação que considera relevante para análise posterior.

Parte A - Instanciação e captura na rede do emulador CORE

Para a validação prática do *sniffer* desenvolvido, foi implementada uma topologia de rede no CORE, permitindo a captura de tráfego num ambiente controlado, verificando corretamente o funcionamento de atividades simples, nomeadamente o ping, traceroute, netcat, funcionamento base do protocolo SSH e ainda a verificação de casos de fragmentação de pacotes.

Topologia de Rede criada

A topologia implementada encontra-se organizada em três segmentos principais, cada um com um papel específico na simulação do ambiente de rede:

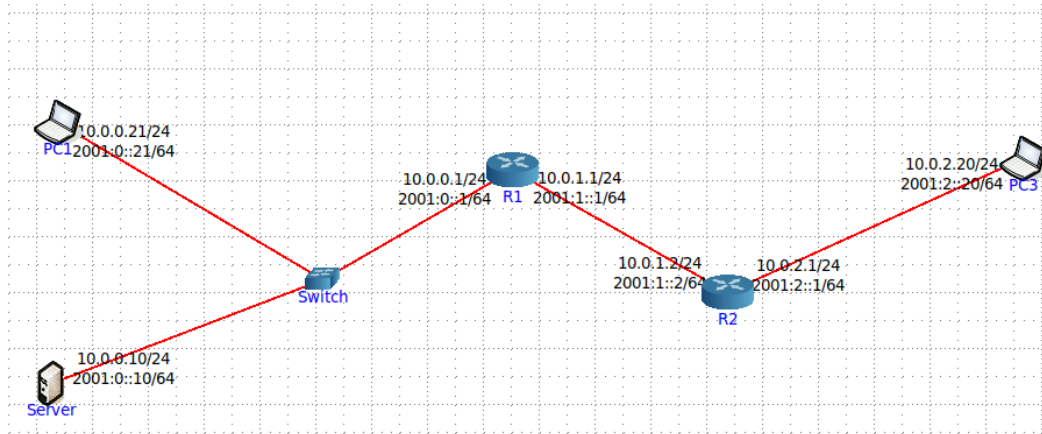
- **Rede Local (LAN):**
Este segmento inclui um *Server*, um *PC1* e um *Switch*, representando uma rede interna típica. O switch assegura a comunicação entre os dispositivos dentro da mesma rede, permitindo simular interações locais como a troca de mensagens ARP e comunicações diretas entre cliente e servidor.
- **Segmento de Interligação (Backbone):**
Constituído pelos routers *R1* e *R2*, este segmento funciona como uma rede de transporte entre diferentes domínios. A sua principal função é encaminhar pacotes entre a rede local e a rede remota, simulando o comportamento de uma infraestrutura intermédia, como a de um ISP.
- **Rede Remota:**
Neste segmento encontra-se o *PC3*, que representa um cliente externo. Este host acede aos serviços disponibilizados pelo *Server*, permitindo simular comunicações que atravessam múltiplos saltos na rede.

A escolha desta topologia não é arbitrária, tendo sido desenhada para cumprir os requisitos definidos na Parte A do projeto:

- Validação ao nível da camada 2 (ARP):
A presença de um *switch* na rede local permite observar de forma simples o processo de resolução de endereços. Antes de qualquer comunicação IP, o *PC1* necessita de descobrir o endereço MAC do *Server*, o que origina a troca de mensagens ARP (*Request* e *Reply*). Este comportamento pode ser facilmente verificado através da execução de um *ping*, sendo possível ao sniffer capturar essas mensagens iniciais.
- Análise de Traceroute (camada 3):
A inclusão de dois routers (*R1* e *R2*) entre o *PC3* e o *Server* cria um cenário adequado para o uso do comando *traceroute*. Isto permite observar a diminuição do valor de TTL em cada salto e a geração de mensagens ICMP do tipo 11 (*Time Exceeded*), fundamentais para mapear o percurso dos pacotes.

- Fluxos de aplicação e transporte (TCP):

Com o sniffer posicionado no servidor, torna-se possível acompanhar o estabelecimento de ligações TCP de forma mais direta, nomeadamente o processo inicial de ligação (three-way handshake). Para além disso, é possível observar dados transportados pelas aplicações (como pedidos simples), confirmando que o sniffer consegue interpretar corretamente informação das camadas superiores, sem necessidade de uma análise de protocolos mais complexos.



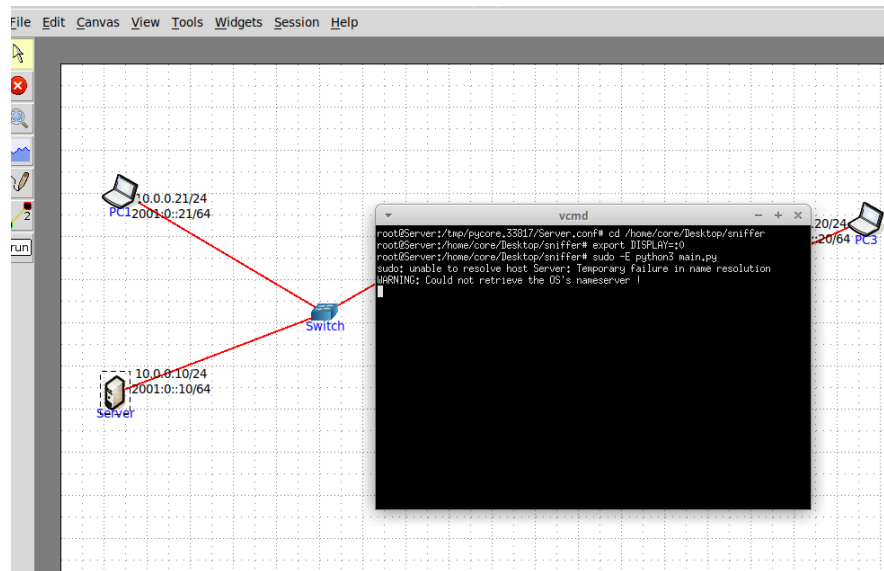
Com a topologia devidamente configurada, procedeu-se à validação funcional do *sniffer*.

A.1 Testes Relativos ao funcionamento do ping

Para verificar o funcionamento da camada de rede, bem como a troca de mensagens segundo o protocolo ARP e a capacidade de resposta da aplicação, foi executado um teste de conectividade simples entre nós da mesma rede local.

A demonstração seguiu os seguintes passos:

1. **Instanciação do sniffer:** O sniffer foi instanciado no Server (10.0.0.10) com privilégios de administrador, usando os comandos:

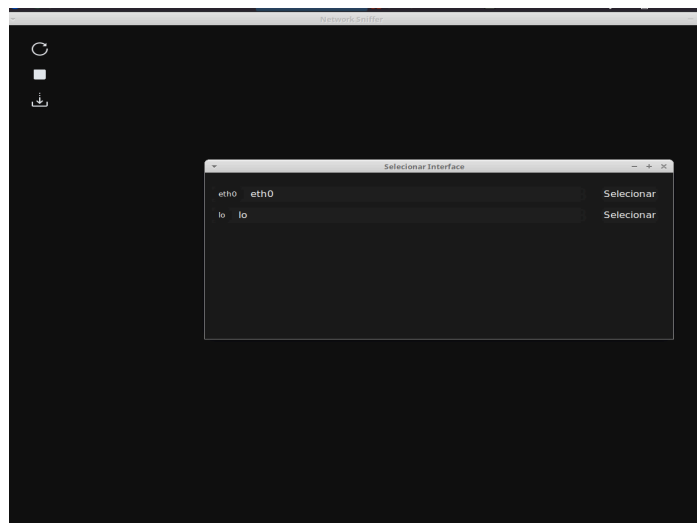


Foi necessário correr:

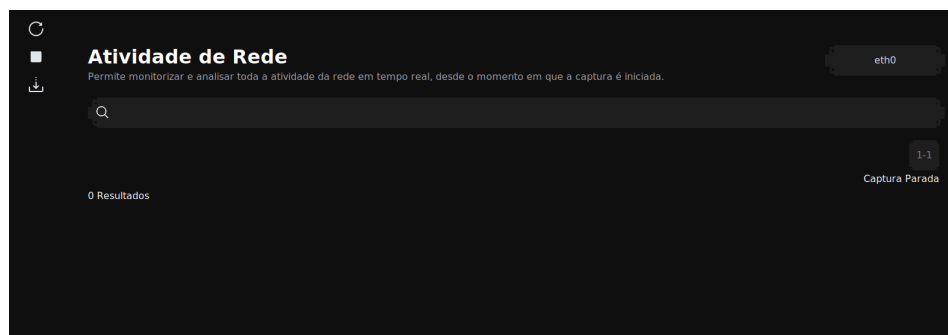
export DISPLAY=:0: Serve para dizer ao nó do CORE que a janela do programa deve ser desenhada no ecrã principal do teu computador, já que os nós da simulação não têm monitor próprio.

sudo -E: É necessário para dar permissões de administrador ao programa (para ele conseguir "ouvir" a rede), mas garantindo que a interface gráfica não bloqueia ao herdar as definições do teu utilizador.

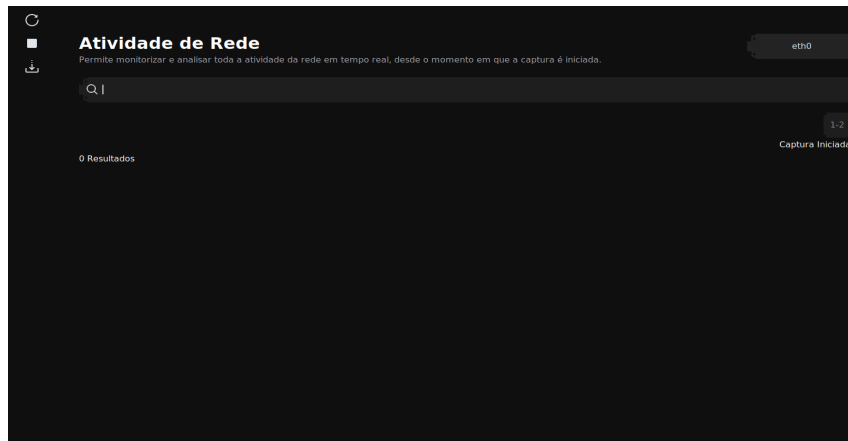
2. **Seleção de Interface:** Na interface gráfica da aplicação, seleccionou-se a interface de rede **eth0**, que liga o servidor ao *switch* da rede local.



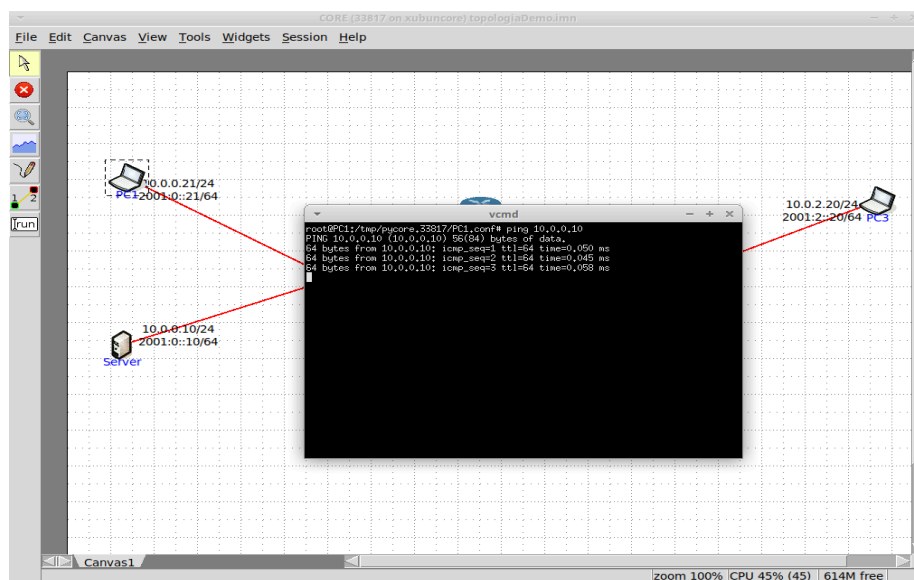
Obtendo:



E de seguida, iniciando a captura:



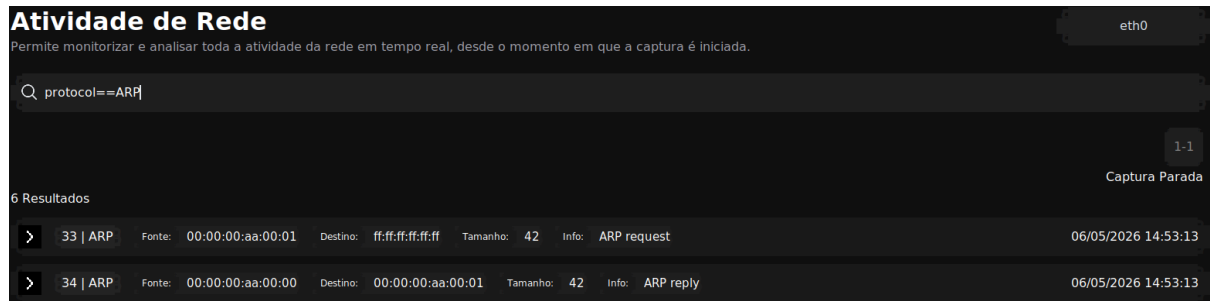
3. Geração de Tráfego: De seguida, no nó PC1 (10.0.0.21), executou-se o comando **ping 10.0.0.10**



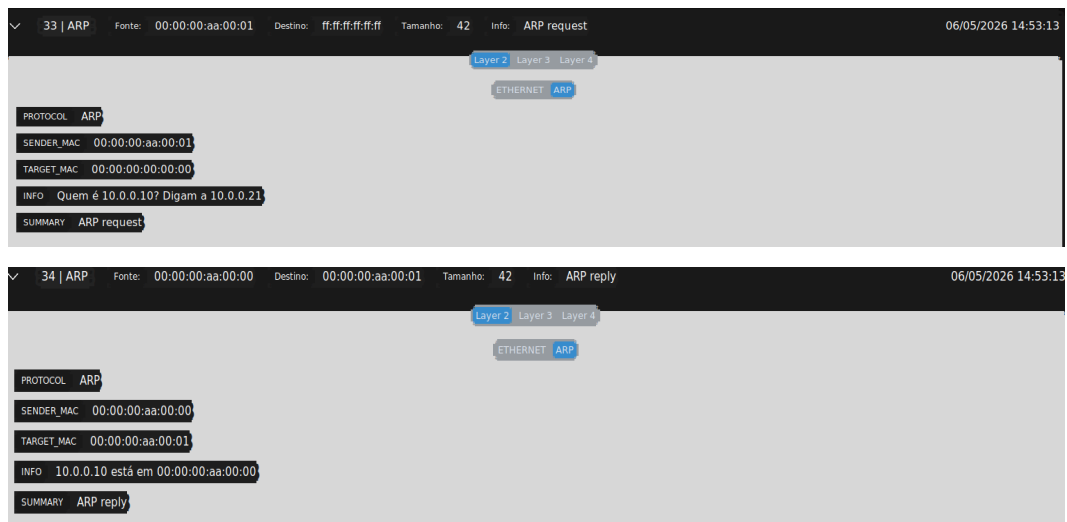
Tendo isto feito, parou-se a captura, obtendo o seguinte resultado:

Atividade de Rede							
Permite monitorizar e analisar toda a atividade da rede em tempo real, desde o momento em que a captura é iniciada.							
Q							
							eth0
							1-14
							Proximo
Captura Iniciada							
140 Resultados							
>	1 IP	Fonte: 10.0.0.1	Destino: 224.0.0.5	Tamanho: 78	Info: Datagrama Normal	06/05/2026 14:52:28	
>	2 IP	Fonte: 10.0.0.1	Destino: 224.0.0.5	Tamanho: 78	Info: Datagrama Normal	06/05/2026 14:52:30	
>	3 IP	Fonte: 10.0.0.1	Destino: 224.0.0.5	Tamanho: 78	Info: Datagrama Normal	06/05/2026 14:52:32	
>	4 IP	Fonte: 10.0.0.1	Destino: 224.0.0.5	Tamanho: 78	Info: Datagrama Normal	06/05/2026 14:52:34	
>	5 IPv6	Fonte: fe80::200:ff:feaa:2	Destino: ff02::5	Tamanho: 90	Info: Datagrama IPv6	06/05/2026 14:52:36	
>	6 IP	Fonte: 10.0.0.1	Destino: 224.0.0.5	Tamanho: 78	Info: Datagrama Normal	06/05/2026 14:52:36	
>	7 IP	Fonte: 10.0.0.1	Destino: 224.0.0.5	Tamanho: 78	Info: Datagrama Normal	06/05/2026 14:52:38	
>	8 IP	Fonte: 10.0.0.1	Destino: 224.0.0.5	Tamanho: 78	Info: Datagrama Normal	06/05/2026 14:52:40	
>	9 IP	Fonte: 10.0.0.1	Destino: 224.0.0.5	Tamanho: 78	Info: Datagrama Normal	06/05/2026 14:52:42	
>	10 IP	Fonte: 10.0.0.1	Destino: 224.0.0.5	Tamanho: 78	Info: Datagrama Normal	06/05/2026 14:52:44	

Tendo sido feita a captura, o próximo passo foi então analisar o tráfego capturado pelo sniffer. Como se pode ver pela imagem acima, foram apanhados pacotes, não só do protocolo ARP e ICMP, mas também pacotes IP e tramas Ethernet, logo para se obter uma visão mais clara, foram aplicados filtros. O primeiro filtro aplicado foi **protocol==ARP** para permitir filtrar pacotes cujo protocolo é o ARP. Desta feita, obteve-se:



Com os cabeçalhos:



Conforme ilustrado nas figuras, o sniffer capturou com sucesso o processo de **resolução de endereços ARP na camada 2**:

- **Pacote 1 (ARP Request):**

O PC1 envia um pedido para a rede perguntando “Quem tem o endereço IP 10.0.0.10?”. Este pacote é enviado em *broadcast*, o que se reflete no endereço MAC de destino Ethernet (*ff:ff:ff:ff:ff:ff*), garantindo que todos os dispositivos da rede local o recebem. No cabeçalho ARP, o campo do target MAC encontra-se preenchido a zeros, uma vez que essa informação ainda é desconhecida. O endereço do sender MAC no cabeçalho ARP, e o MAC src do cabeçalho Ethernet, correspondem ao endereço MAC do dispositivo que envia o pedido (PC1).

- **Pacote 2 (ARP Reply):**

O servidor responde diretamente ao PC1, fornecendo o seu endereço MAC (*00:00:00:aa:00:00*). O sniffer consegue capturar e interpretar corretamente esta resposta, apresentando-a de forma legível: “10.0.0.10 está em 00:00:00:aa:00:00”. Ao contrário do pedido, esta mensagem é enviada em *unicast*, uma vez que o servidor já conhece o endereço MAC do PC1. Isto acontece porque, ao receber o

ARP Request, o servidor obtém o mapeamento entre o IP e o MAC do PC1 e guarda essa informação na sua tabela ARP.

Isto é possível devido ao processamento de pacotes ARP implementado no sniffer. Ao analisar cada pacote, o programa verifica o tipo de operação presente no campo **arp.op**, distinguindo entre **ARP Request** (valor 1) e **ARP Reply** (valor 2). Com base nessa identificação, o sniffer constrói dinamicamente a informação a apresentar ao utilizador. No caso de um *request*, é gerada uma mensagem que indica qual o endereço IP que está a ser procurado e quem fez o pedido. Já no caso de um *reply*, é apresentada a associação entre o endereço IP (**arp.psrc**) e o respetivo endereço MAC (**arp.hwsrc**). Desta forma, o sniffer não só captura os pacotes, como também interpreta os seus campos relevantes, convertendo-os numa representação mais legível e informativa para o utilizador

```
def ARPProcesser(packet: Packet):
    if packet.haslayer(ARP):
        arp = packet[ARP]

        if arp.op == 1:
            summary = "ARP request"
            info = f"Quem é {arp.pdst}? Digam a {arp.psrc}"
        elif arp.op == 2:
            summary = "ARP reply"
            info = f"{arp.psrc} está em {arp.hwsrc}"
        else:
            summary = f"ARP op {arp.op}"
            info = "Protocolo de controlo ARP"

        return {
            "protocol": "ARP",
            "sender_mac": arp.hwsrc,
            "target_mac": arp.hwdst,
            "info": info,
            "summary": summary
        }
    return None
```

Após, foi aplicado o filtro **protocol==ICMP** de modo a fazer display apenas dos pacotes ICMP capturados durante a captura, fruto do uso do ping.

Atividade de Rede									
Permite monitorizar e analisar toda a atividade da rede em tempo real, desde o momento em que a captura é iniciada.									
Q protocol==ICMP									
1-6 Proximo									
Captura Parada									
60 Resultados									
>	35	ICMP	Fonte: 10.0.0.21	Destino: 10.0.0.10	Tamanho: 98	Info: ICMP echo request	06/05/2026 14:53:13		
>	36	ICMP	Fonte: 10.0.0.10	Destino: 10.0.0.21	Tamanho: 98	Info: ICMP echo reply	06/05/2026 14:53:13		
>	38	ICMP	Fonte: 10.0.0.21	Destino: 10.0.0.10	Tamanho: 98	Info: ICMP echo request	06/05/2026 14:53:14		
>	39	ICMP	Fonte: 10.0.0.10	Destino: 10.0.0.21	Tamanho: 98	Info: ICMP echo reply	06/05/2026 14:53:14		
>	40	ICMP	Fonte: 10.0.0.21	Destino: 10.0.0.10	Tamanho: 98	Info: ICMP echo request	06/05/2026 14:53:15		
>	41	ICMP	Fonte: 10.0.0.10	Destino: 10.0.0.21	Tamanho: 98	Info: ICMP echo reply	06/05/2026 14:53:15		
>	44	ICMP	Fonte: 10.0.0.21	Destino: 10.0.0.10	Tamanho: 98	Info: ICMP echo request	06/05/2026 14:53:16		
>	45	ICMP	Fonte: 10.0.0.10	Destino: 10.0.0.21	Tamanho: 98	Info: ICMP echo reply	06/05/2026 14:53:16		
>	46	ICMP	Fonte: 10.0.0.21	Destino: 10.0.0.10	Tamanho: 98	Info: ICMP echo request	06/05/2026 14:53:17		
>	47	ICMP	Fonte: 10.0.0.10	Destino: 10.0.0.21	Tamanho: 98	Info: ICMP echo reply	06/05/2026 14:53:17		

Analisando a figura verifica-se a captura de 10 pacotes (5 pares ICMP echo request - ICMP echo reply), correspondendo ao que foi possível capturar enquanto a captura esteve ativa. Analisando as mensagens ICMP podemos concluir que a aplicação registou o funcionamento típico do ping, verificando-se que:

- O host onde foi executado o comando, o **PC1 (IP 10.0.0.21)**, envia sucessivos pacotes **ICMP Echo Request** com destino ao **Server (IP 10.0.0.10)**, conforme indicado no comando executado.
- Como resposta, o servidor envia mensagens **ICMP Echo Reply**, cujo endereço de origem (*source*) é o seu próprio IP e o destino é o **IP 10.0.0.21** correspondente ao **PC1**.

Podemos ainda confirmar que todas as mensagens **ICMP Echo Request** são enviadas através do IP source **10.0.0.21** (interface de PC1) aplicando o filtro **protocol==ICMP and src==10.0.0.21**, e que as mensagens **ICMP Echo Reply** são enviadas através do IP source **10.0.0.10** (interface do Server), aplicando o filtro **protocol==ICMP and src==10.0.0.10**, tal como se ilustra nas seguintes imagens, respetivamente:

The top screenshot shows the Wireshark interface with the filter `protocol==ICMP and src==10.0.0.21`. It displays five ICMP echo request packets from source 10.0.0.21 to destination 10.0.0.10. The bottom screenshot shows the Wireshark interface with the filter `protocol==ICMP and src==10.0.0.10`. It displays five ICMP echo reply packets from source 10.0.0.10 to destination 10.0.0.21.

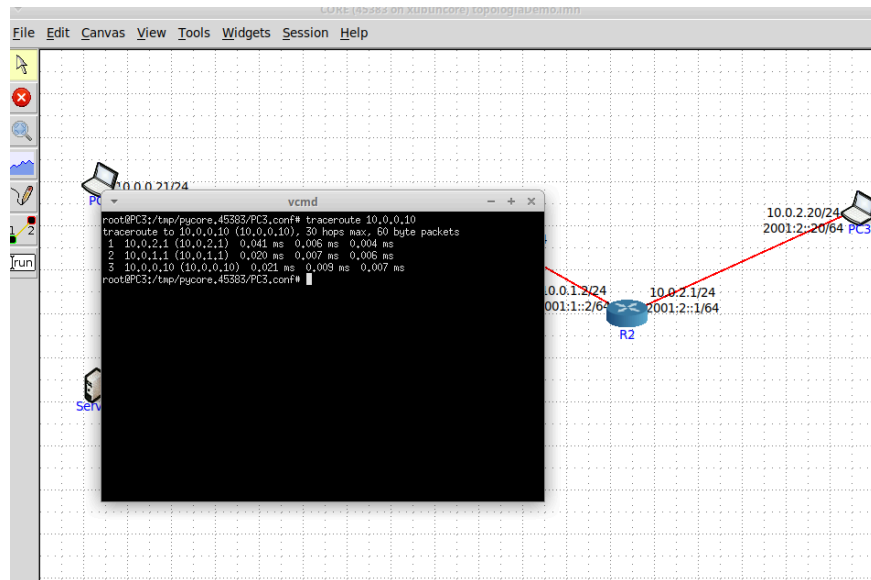
No.	Time	Source	Destination	Length	Info
35	06/05/2026 14:53:13	10.0.0.21	10.0.0.10	98	ICMP echo request
38	06/05/2026 14:53:14	10.0.0.21	10.0.0.10	98	ICMP echo request
40	06/05/2026 14:53:15	10.0.0.21	10.0.0.10	98	ICMP echo request
44	06/05/2026 14:53:16	10.0.0.21	10.0.0.10	98	ICMP echo request
46	06/05/2026 14:53:17	10.0.0.21	10.0.0.10	98	ICMP echo request

No.	Time	Source	Destination	Length	Info
36	06/05/2026 14:53:13	10.0.0.10	10.0.0.21	98	ICMP echo reply
39	06/05/2026 14:53:14	10.0.0.10	10.0.0.21	98	ICMP echo reply
41	06/05/2026 14:53:15	10.0.0.10	10.0.0.21	98	ICMP echo reply
45	06/05/2026 14:53:16	10.0.0.10	10.0.0.21	98	ICMP echo reply
47	06/05/2026 14:53:17	10.0.0.10	10.0.0.21	98	ICMP echo reply

A.2 Testes Relativos ao funcionamento do traceroute

Para validar o comportamento do Traceroute, a captura foi realizada no PC3 (10.0.2.20), que funciona como origem do tráfego. Esta escolha justifica-se pelo facto de ser este o nó onde se consegue observar todo o processo, incluindo os pacotes enviados e as respostas de erro dos routers intermédios. O processo de instanciação do sniffer, seleção de interface e início de captura no no hOst PC3 é idêntico ao realizado nas secção anterior.

Tendo sido iniciada a captura, procedeu-se então à invocação, numa outra shell do host PC3, do comando: **traceroute 10.0.0.10**. O comando é invocado desta forma de modo a ter como destino dos pacotes enviados o host Server, e não é passada flag nenhuma de modo a permitir capturar pacotes UDP, no entanto seria possível passar a flag **-I**, de modo a serem enviados pacotes ICMP (ver mais à frente).



Tendo sido corrido o comando `traceroute`, fomos à interface do nosso sniffer e paramos a captura. Dentro do tráfego obtido, decidimos focar a nossa atenção na troca de pacotes UDP e ICMP e por isso aplicamos o filtro **protocol==UDP or protocol==ICMP**, obtendo:

Atividade de Rede							eth0
Permite monitorizar e analisar toda a atividade da rede em tempo real, desde o momento em que a captura é iniciada.							
Q protocol==UDP or protocol==ICMP							
							1-3 Proximo
28 Resultados							Captura Parada
>	4 UDP	Fonte: 10.0.2.20	Destino: 10.0.0.10	Tamanho: 74	Info: UDP 46191 → 33434		06/05/2026 15:04:10
>	5 ICMP	Fonte: 10.0.2.1	Destino: 10.0.2.20	Tamanho: 102	Info: Time exceeded		06/05/2026 15:04:10
>	6 UDP	Fonte: 10.0.2.20	Destino: 10.0.0.10	Tamanho: 74	Info: UDP 55436 → 33435		06/05/2026 15:04:10
>	7 ICMP	Fonte: 10.0.2.1	Destino: 10.0.2.20	Tamanho: 102	Info: Time exceeded		06/05/2026 15:04:10
>	8 UDP	Fonte: 10.0.2.20	Destino: 10.0.0.10	Tamanho: 74	Info: UDP 36411 → 33436		06/05/2026 15:04:10
>	9 ICMP	Fonte: 10.0.2.1	Destino: 10.0.2.20	Tamanho: 102	Info: Time exceeded		06/05/2026 15:04:10
>	10 UDP	Fonte: 10.0.2.20	Destino: 10.0.0.10	Tamanho: 74	Info: UDP 52688 → 33437		06/05/2026 15:04:10
>	11 ICMP	Fonte: 10.0.1.1	Destino: 10.0.2.20	Tamanho: 102	Info: Time exceeded		06/05/2026 15:04:10
>	12 UDP	Fonte: 10.0.2.20	Destino: 10.0.0.10	Tamanho: 74	Info: UDP 40663 → 33438		06/05/2026 15:04:10
>	13 ICMP	Fonte: 10.0.1.1	Destino: 10.0.2.20	Tamanho: 102	Info: Time exceeded		06/05/2026 15:04:10

Analisando o tráfego concretamente, foi possível verificar, através do filtro: **protocol==UDP and layer3.ip.ttl==1**, observando-se a captura de pacotes UDP emitidos com ttl = 1.

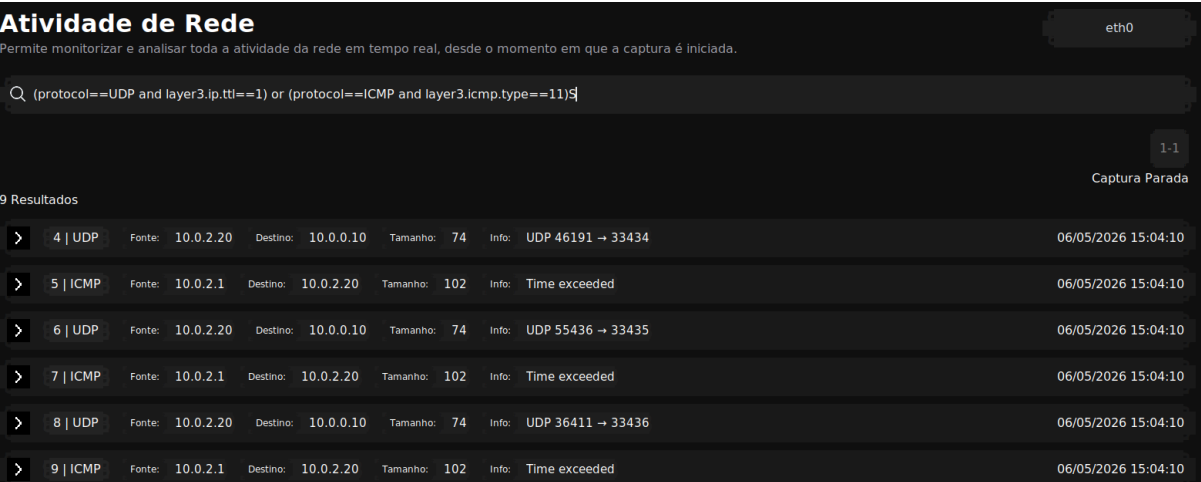


	4 UDP	Fonte: 10.0.2.20	Destino: 10.0.0.10	Tamanho: 74	Info: UDP 46191 → 33434	06/05/2026 15:04:10
>	6 UDP	Fonte: 10.0.2.20	Destino: 10.0.0.10	Tamanho: 74	Info: UDP 55436 → 33435	06/05/2026 15:04:10
>	8 UDP	Fonte: 10.0.2.20	Destino: 10.0.0.10	Tamanho: 74	Info: UDP 36411 → 33436	06/05/2026 15:04:10

Como se pode verificar, todos os pacotes UDP enviados pelo *traceroute* têm como endereço IP de origem o host *PC3* (10.0.2.20), uma vez que é neste equipamento que o comando foi executado. Analisando o cabeçalho da camada 3, observa-se que o campo **TTL (Time To Live)** inicial dos primeiros pacotes é igual a 1.

Relativamente ao tamanho dos datagramas, verifica-se que estes possuem 60 bytes, valor observado no campo **LEN** do cabeçalho IP, enquanto o valor apresentado no *summary* (74 bytes) inclui também o cabeçalho da camada 2 (Ethernet), refletindo assim o tamanho total do frame capturado. Este tamanho é verificado no output do comando *traceroute* na shell de *PC3* (onde foi corrido)

Quando estes pacotes com TTL igual a 1 chegam ao primeiro router da topologia (*R2*), o valor do TTL é decrementado até zero, levando ao descarte do pacote. Como resposta, o router gera mensagens ICMP do tipo **Time Exceeded**, indicando que o TTL expirou. Para poder filtrar, tanto os pacotes UDP com TTL = 1, mas ver também os devidos pacotes ICMP de resposta, aplicou-se o filtro (**protocol==UDP and layer3.ip.ttl==1**) or (**protocol==ICMP and layer3.icmp.type==11**), obtendo:



	4 UDP	Fonte: 10.0.2.20	Destino: 10.0.0.10	Tamanho: 74	Info: UDP 46191 → 33434	06/05/2026 15:04:10
>	5 ICMP	Fonte: 10.0.2.1	Destino: 10.0.2.20	Tamanho: 102	Info: Time exceeded	06/05/2026 15:04:10
>	6 UDP	Fonte: 10.0.2.20	Destino: 10.0.0.10	Tamanho: 74	Info: UDP 55436 → 33435	06/05/2026 15:04:10
>	7 ICMP	Fonte: 10.0.2.1	Destino: 10.0.2.20	Tamanho: 102	Info: Time exceeded	06/05/2026 15:04:10
>	8 UDP	Fonte: 10.0.2.20	Destino: 10.0.0.10	Tamanho: 74	Info: UDP 36411 → 33436	06/05/2026 15:04:10
>	9 ICMP	Fonte: 10.0.2.1	Destino: 10.0.2.20	Tamanho: 102	Info: Time exceeded	06/05/2026 15:04:10

As mensagens podem ser identificadas pelo endereço IP de origem, que corresponde ao router (10.0.2.1), e pelo endereço IP de destino, que corresponde ao host *PC3* (10.0.2.20).

Adicionalmente, observa-se que o campo TTL destas mensagens ICMP apresenta o valor 64, que corresponde ao valor padrão definido no sistema operativo do router, e não tem relação direta com o mecanismo de expiração do TTL dos pacotes originalmente enviados pelo *traceroute*. A título de exemplo, mostram-se para verificar estas afirmações, os cabeçalhos de um dos pacotes UDP, e de uma das mensagens ICMP

4 | UDP

Fonte: 10.0.2.20 Destino: 10.0.0.10 Tamanho: 74 Info: UDP 46191 → 33434

06/05/2026 15:04:10

Layer 2 Layer 3 Layer 4

IP

VERSION 4

HEADER_LENGTH 5

LEN 60

ID 45282

FLAGS 0

OFFSET 0

PROTOCOL 17

TTL 1

SRC 10.0.2.20

DST 10.0.0.10

SUMMARY Datagrama Normal

5 | ICMP

Fonte: 10.0.2.1 Destino: 10.0.2.20 Tamanho: 102 Info: Time exceeded

06/05/2026 15:04:10

Layer 2 Layer 3 Layer 4

IP ICMP

VERSION 4

HEADER_LENGTH 5

LEN 88

ID 16871

FLAGS 0

OFFSET 0

PROTOCOL 1

TTL 64

SRC 10.0.2.1

DST 10.0.2.20

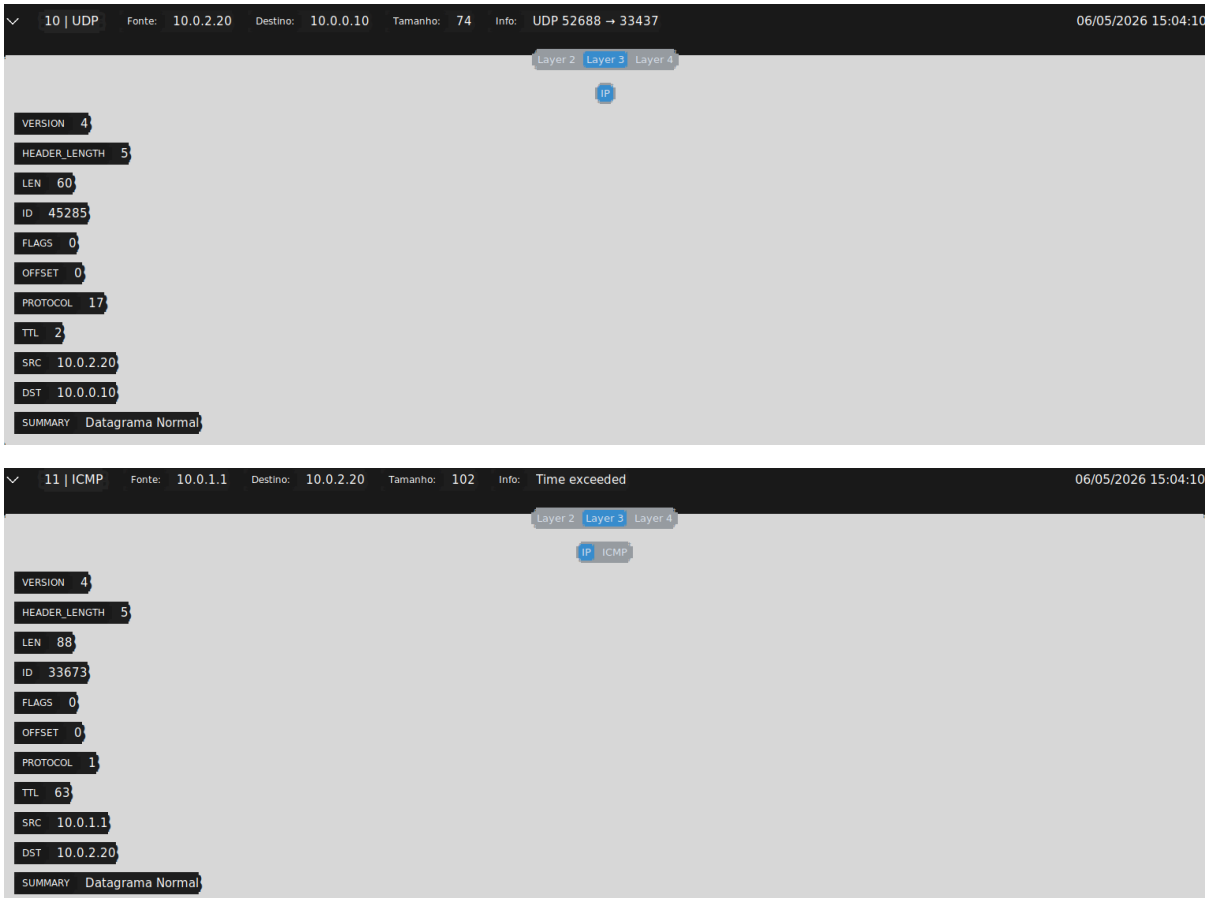
SUMMARY Datagrama Normal

atravessarem o router *R2*, o TTL é decrementado para 1, valor ainda suficiente para o pacote continuar a sua trajetória na rede. No entanto, ao chegar ao router *R1*, o TTL é novamente decrementado, atingindo o valor 0. Como consequência, o pacote é descartado e o router gera uma mensagem ICMP *Time Exceeded* em resposta. Este comportamento permite identificar o segundo salto na topologia. Na análise dos pacotes capturados, observa-se que os novos datagramas UDP continuam a ter como endereço IP de origem o host *PC3* (10.0.2.20) e como destino o host *Server* (10.0.0.10), uma vez que o objetivo do *traceroute* é atingir esse destino final.

No caso das mensagens ICMP geradas, o endereço IP de origem corresponde à interface do router *R1* (10.0.1.1), enquanto o destino é o host *PC3* (10.0.2.20)

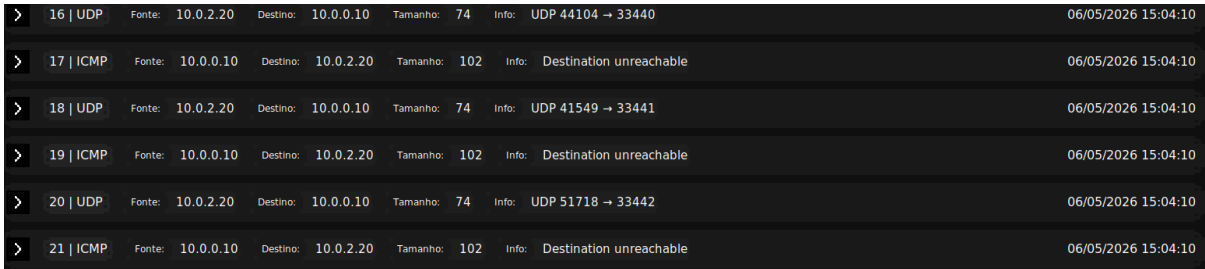
>	10 UDP	Fonte: 10.0.2.20	Destino: 10.0.0.10	Tamanho: 74	Info: UDP 52688 → 33437	06/05/2026 15:04:10
>	11 ICMP	Fonte: 10.0.1.1	Destino: 10.0.2.20	Tamanho: 102	Info: Time exceeded	06/05/2026 15:04:10
>	12 UDP	Fonte: 10.0.2.20	Destino: 10.0.0.10	Tamanho: 74	Info: UDP 40663 → 33438	06/05/2026 15:04:10
>	13 ICMP	Fonte: 10.0.1.1	Destino: 10.0.2.20	Tamanho: 102	Info: Time exceeded	06/05/2026 15:04:10
>	14 UDP	Fonte: 10.0.2.20	Destino: 10.0.0.10	Tamanho: 74	Info: UDP 33131 → 33439	06/05/2026 15:04:10
>	15 ICMP	Fonte: 10.0.1.1	Destino: 10.0.2.20	Tamanho: 102	Info: Time exceeded	06/05/2026 15:04:10

É possível ainda verificar que efetivamente as mensagens UDP foram enviadas com um TTL = 2 (TTL anterior (1) + 1), analisando o seu cabeçalho. Verifica-se ainda que o TTL da mensagem ICMP com id 9 é também elevado (63), devido à explicação fornecida anteriormente.

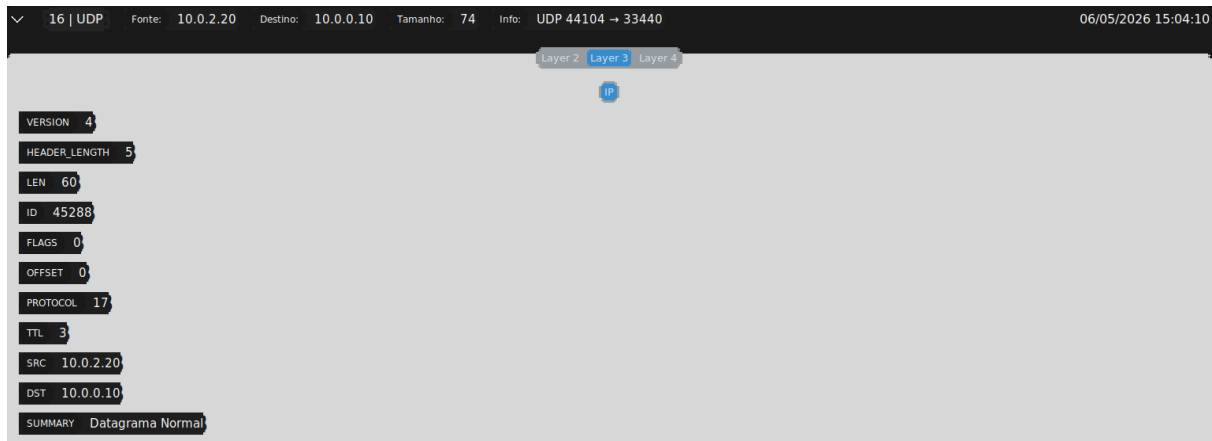


De seguida o traceroute envia mais 3 pacotes UDP, agora com TTL = 3. Este valor de TTL já é suficientemente alto para que nenhum router descarte o pacote, e por isso o pacote consegue efetivamente atingir o host Server. Como resposta o host Server envia uma mensagem do ICMP Destination Unreachable, indicando portanto ao traceroute que pode parar a emissão de pacotes uma vez que o destino foi atingido.

É possível verificar isto na seguinte figura:

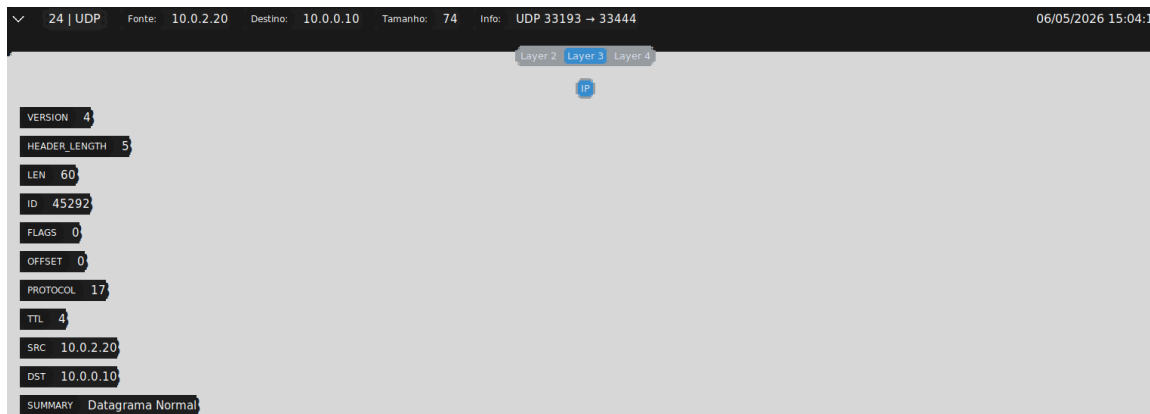


Mais uma vez, de modo a comprovar que o TTL dos pacotes UDP enviados é 3, podemos verificar essa afirmação conferindo o cabeçalho IP dos pacotes, como por exemplo, do pacote com o id 16:

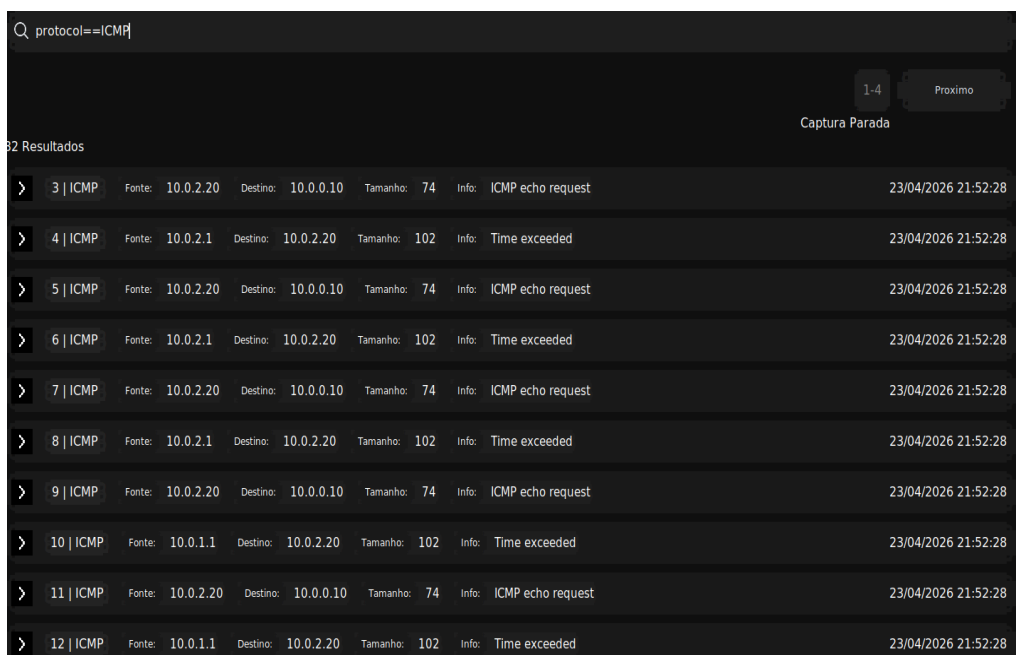


Por último, foram ainda capturados alguns pacotes UDP com TTL = 4, 5 e ainda 6, isto porque o traceroute quando recebe a mensagem ICMP Destination Unreachable não termina instantaneamente a sua execução e por isso ainda são emitidos alguns pacotes.

Pacote com TTL=4.



O comportamento do traceroute podia ser também analisado, passando a flag -I no comando, o que até tornaria mais fácil o filtro. Desta forma, correndo o traceroute para o destino 10.0.0.10 com a flag -I comando numa shell do host PC3, obtendo o seguinte tráfego:



O comportamento é bastante semelhante ao do traceroute sem a flag, simplesmente as mensagens UDP foram “troçadas” por mensagens ICMP Echo Request e as mensagens enviadas pelo server quando os pacotes têm TTL suficiente para lá chegar são do tipo ICMP Echo Reply.

A.3 Testes de Transferência Simples (TCP/Netcat)

De seguida, utilizou-se a ferramenta **Netcat** para simular uma transferência de dados entre o **PC1 (10.0.0.21)** e o **Server (10.0.0.10)**. O objetivo foi validar a capacidade do *sniffer* em monitorizar fluxos de transporte TCP, identificando o estabelecimento, a transferência de informação e o encerramento da sessão.

Procedimento: O processo de instanciação do *sniffer* seguiu o método descrito nas secções anteriores, sendo aplicado na interface **eth0** do servidor. Tendo a captura ativa, o teste foi executado nos seguintes passos:

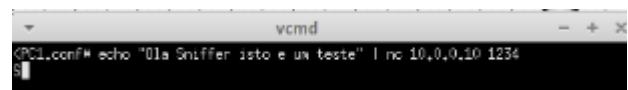
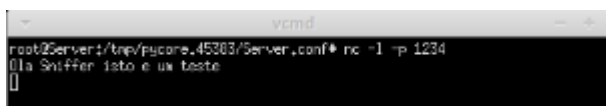
1. No **Server**, o Netcat foi colocado em modo de escuta na porta 1234

nc -l -p 1234

2. No **PC1**, foi estabelecida a ligação e enviada uma mensagem de texto:

echo "Ola Sniffer isto e um teste" | nc 10.0.0.10 1234

3. Após a confirmação da receção, a ligação foi encerrada manualmente no PC1 através de um sinal de interrupção (Ctrl+C).



De seguida parou-se a captura, e aplicamos o filtro **protocol==TCP** de modo a filtrar os pacotes do protocolo TCP, obtendo os seguintes resultados:

Atividade de Rede									
Permite monitorizar e analisar toda a atividade da rede em tempo real, desde o momento em que a captura é iniciada.									
Q protocol==TCP									
1-1									
Captura Parada									
8 Resultados									
>	4 TCP	Fonte: 10.0.0.21	Destino: 10.0.0.10	Tamanho: 74	Info: TCP SYN 34876 → 1234	23/04/2026 22:19:13			
>	5 TCP	Fonte: 10.0.0.10	Destino: 10.0.0.21	Tamanho: 74	Info: TCP SYN-ACK 1234 → 34876	23/04/2026 22:19:13			
>	6 TCP	Fonte: 10.0.0.21	Destino: 10.0.0.10	Tamanho: 66	Info: TCP ACK 34876 → 1234	23/04/2026 22:19:13			
>	7 TCP	Fonte: 10.0.0.21	Destino: 10.0.0.10	Tamanho: 94	Info: TCP 34876 → 1234	23/04/2026 22:19:13			
>	8 TCP	Fonte: 10.0.0.10	Destino: 10.0.0.21	Tamanho: 66	Info: TCP ACK 1234 → 34876	23/04/2026 22:19:13			
>	51 TCP	Fonte: 10.0.0.21	Destino: 10.0.0.10	Tamanho: 66	Info: TCP 34876 → 1234	23/04/2026 22:20:16			
>	52 TCP	Fonte: 10.0.0.10	Destino: 10.0.0.21	Tamanho: 66	Info: TCP 1234 → 34876	23/04/2026 22:20:16			
>	53 TCP	Fonte: 10.0.0.21	Destino: 10.0.0.10	Tamanho: 66	Info: TCP ACK 34876 → 1234	23/04/2026 22:20:16			

Análise de Resultados: A captura obtida permite observar as três fases distintas do ciclo de vida de uma ligação TCP, comprovando a precisão e a capacidade de análise do *sniffer* desenvolvido:

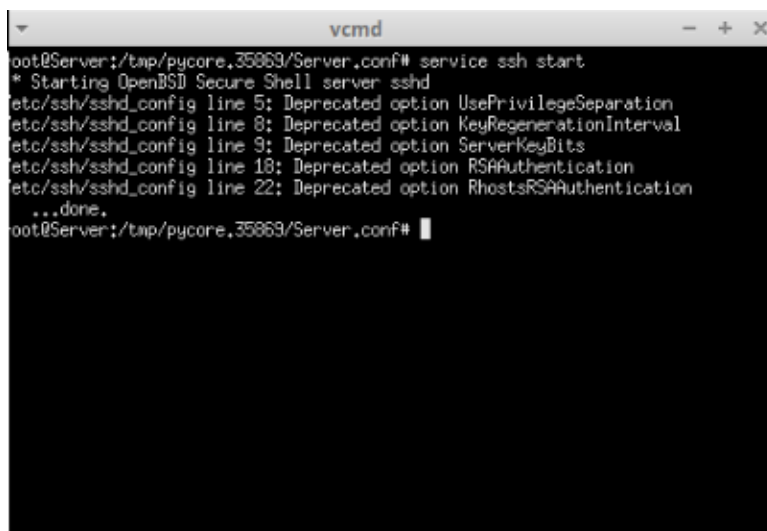
- **Estabelecimento de Ligação (Handshake):** Através dos pacotes com os **IDs 4 a 6**, o *sniffer* regista sequencialmente a troca de flags **SYN**, **SYN-ACK** e **ACK**. Esta sequência demonstra que a aplicação identifica corretamente a negociação inicial entre o PC1 e o Server, garantindo que a comunicação foi estabelecida de forma fiável.
- **Transferência de Dados:** O pacote com o **ID 7** representa o envio efetivo de informação pelo PC1. O aumento do seu tamanho para **94 bytes**, comparativamente aos pacotes de controlo anteriores, confirma o transporte do *payload* de aplicação. Logo de seguida, o pacote com o **ID 8** mostra o servidor a confirmar a receção dos dados através de um **ACK**.
- **Encerramento Ordenado:** Após a interrupção manual no cliente (**Ctrl+C**), o *sniffer* capturou a sequência de finalização entre os pacotes com os **IDs 51 e 53**. A capacidade de registar este encerramento é crucial, pois valida que a ferramenta consegue monitorizar a libertação de recursos da rede e o fecho correto das instâncias de comunicação.

A.4 Monitorização de Protocolo de Acesso Remoto (SSH) – PC3 para Server

Para validar a robustez do *sniffer* na identificação de serviços padrão e a sua capacidade de processar tráfego proveniente de diferentes sub-redes, realizou-se uma ligação **SSH** (Secure Shell) entre o **PC3 (10.0.2.20)** e o **Server (10.0.0.10)**.

Procedimento: Mantendo a instância do *sniffer* na interface **eth0** do servidor, o teste de acesso remoto foi executado seguindo estes passos:

1. No **Server**, garantiu-se que o serviço SSH estava ativo para receber ligações, utilizando o comando **service ssh start**
2. No **PC3**, iniciou-se a tentativa de ligação ao servidor através do comando: **ssh root@10.0.0.10**
3. Após a troca inicial de pacotes e a negociação de chaves, a sessão foi interrompida manualmente para analisar o comportamento do fluxo.



```
vcmd
root@Server:/tmp/pycore.35869/Server.conf# service ssh start
* Starting OpenBSD Secure Shell server: sshd
etc/ssh/sshd_config line 5: Deprecated option UsePrivilegeSeparation
etc/ssh/sshd_config line 8: Deprecated option KeyRegenerationInterval
etc/ssh/sshd_config line 9: Deprecated option ServerKeyBits
etc/ssh/sshd_config line 18: Deprecated option RSAAuthentication
etc/ssh/sshd_config line 22: Deprecated option RhostsRSAAuthentication
...done.
root@Server:/tmp/pycore.35869/Server.conf#
```

```

root@PC3:/tmp/pycore.35869/PC3.conf# ssh root@10.0.0.10
The authenticity of host '10.0.0.10 (10.0.0.10)' can't be established.
RSA key fingerprint is SHA256:dRFvfn6NujSeht8zCP1wly/Q2fjt4V6XRzpwvpj9TW.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.0.0.10' (RSA) to the list of known hosts.
root@10.0.0.10's password:
Permission denied, please try again.
root@10.0.0.10's password:
Permission denied, please try again.
root@10.0.0.10's password:
Welcome to Ubuntu 20.04.3 LTS (GNU/Linux 5.11.0-37-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

504 updates can be applied immediately.
503 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

Your Hardware Enablement Stack (HWE) is supported until April 2025.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
root@Server:~#

```

De seguida, aplicou-se o filtro **protocol==SSH** na interface da aplicação (que filtra internamente os pacotes TCP com destino ou origem na porta 22, obtendo os seguintes resultados:

Q protocol==SSH

1-6

Proximo

Captura Parada

60 Resultados

>

60 | SSH

Fonte: 10.0.2.20

Destino: 10.0.0.10

Tamanho: 74

Info: TCP SYN (SSH)

23/04/2026 22:52:51

>

61 | SSH

Fonte: 10.0.0.10

Destino: 10.0.2.20

Tamanho: 74

Info: TCP SYN-ACK (SSH)

23/04/2026 22:52:51

>

62 | SSH

Fonte: 10.0.2.20

Destino: 10.0.0.10

Tamanho: 66

Info: TCP ACK (SSH)

23/04/2026 22:52:51

>

63 | SSH

Fonte: 10.0.2.20

Destino: 10.0.0.10

Tamanho: 107

Info: TCP (SSH)

23/04/2026 22:52:51

>

64 | SSH

Fonte: 10.0.0.10

Destino: 10.0.2.20

Tamanho: 66

Info: TCP ACK (SSH)

23/04/2026 22:52:51

>

65 | SSH

Fonte: 10.0.0.10

Destino: 10.0.2.20

Tamanho: 107

Info: TCP (SSH)

23/04/2026 22:52:51

>

66 | SSH

Fonte: 10.0.2.20

Destino: 10.0.0.10

Tamanho: 66

Info: TCP ACK (SSH)

23/04/2026 22:52:51

>

67 | SSH

Fonte: 10.0.2.20

Destino: 10.0.0.10

Tamanho: 1514

Info: TCP ACK (SSH)

23/04/2026 22:52:51

>

68 | SSH

Fonte: 10.0.0.10

Destino: 10.0.2.20

Tamanho: 66

Info: TCP ACK (SSH)

23/04/2026 22:52:51

>

69 | SSH

Fonte: 10.0.2.20

Destino: 10.0.0.10

Tamanho: 130

Info: TCP (SSH)

23/04/2026 22:52:51

Análise de Resultados: A captura obtida revela a complexidade inerente a um protocolo de acesso seguro, destacando-se os seguintes pontos:

- **Sincronização Inicial (IDs 60 a 62):** O *sniffer* identificou com precisão o *Three-Way Handshake* na porta 22. É relevante notar que a etiqueta (**SSH**) é atribuída imediatamente no pacote de **SYN (ID 60)**, demonstrando que a lógica de inspeção de portas *well-known* está a funcionar em tempo real.

- **Negociação de Protocolo e Payload (IDs 63, 65 e 69):** Após o estabelecimento da ligação, observam-se pacotes de dados identificados apenas como **TCP (SSH)**. Estes pacotes contêm a negociação de versões e a troca de chaves criptográficas.

A.5 – Validação de Fragmentação de Pacotes IP

Para testar a capacidade da aplicação em lidar com pacotes que excedem o MTU padrão da rede (1500 bytes), forçou-se a fragmentação na camada de rede utilizando o protocolo ICMP. Para isto, o sniffer foi novamente instanciado no host server.

Procedimento:

1. No **PC1**, executou-se um comando de *ping* com um *payload* de **4000 bytes**: **ping -s 4000 10.0.0.10**.
2. O protocolo IP, ao detetar que o pacote excedia a capacidade da interface física, dividiu-o em três fragmentos.
3. Para conseguir detetar melhor esta fragmentação, sabendo que os pacotes enviados pelo ping (com IP src a ser a interface de PC1, 10.0.0.21), aplicou-se um filtro por IP src, de modo a facilitar a análise: **src==10.0.0.21**, obtendo os seguintes resultados:

Atividade de Rede							eth0
Permite monitorizar e analisar toda a atividade da rede em tempo real, desde o momento em que a captura é iniciada.							
Q packet_id>1							
							1-9 Proximo
84 Resultados							Captura Parada
>	2 ICMP	Fonte: 10.0.0.21	Destino: 10.0.0.10	Tamanho: 1514	Info: ICMP echo request		06/05/2026 15:34:47
>	3 IP	Fonte: 10.0.0.21	Destino: 10.0.0.10	Tamanho: 1514	Info: Datagrama Fragmentado		06/05/2026 15:34:47
>	4 IP	Fonte: 10.0.0.21	Destino: 10.0.0.10	Tamanho: 1082	Info: Datagrama Fragmentado		06/05/2026 15:34:47
>	5 ICMP	Fonte: 10.0.0.10	Destino: 10.0.0.21	Tamanho: 1514	Info: ICMP echo reply		06/05/2026 15:34:47
>	6 IP	Fonte: 10.0.0.10	Destino: 10.0.0.21	Tamanho: 1514	Info: Datagrama Fragmentado		06/05/2026 15:34:47
>	7 IP	Fonte: 10.0.0.10	Destino: 10.0.0.21	Tamanho: 1082	Info: Datagrama Fragmentado		06/05/2026 15:34:47
>	8 IPv6	Fonte: fe80::200:ff:feaa:2	Destino: ff02::5	Tamanho: 90	Info: Datagrama IPv6		06/05/2026 15:34:48
>	9 ICMP	Fonte: 10.0.0.21	Destino: 10.0.0.10	Tamanho: 1514	Info: ICMP echo request		06/05/2026 15:34:48
>	10 IP	Fonte: 10.0.0.21	Destino: 10.0.0.10	Tamanho: 1514	Info: Datagrama Fragmentado		06/05/2026 15:34:48
>	11 IP	Fonte: 10.0.0.21	Destino: 10.0.0.10	Tamanho: 1082	Info: Datagrama Fragmentado		06/05/2026 15:34:48

Análise de Resultados:

Focando a análise nas primeiras 3 entradas (IDs 1, 2 e 3), uma vez que nas restantes o processo é idêntico, verificamos que o *sniffer* interpretou corretamente a fragmentação do datagrama IP:

- **Identificação de Protocolo e Camada (ID 2):** O primeiro fragmento é classificado como **ICMP**, pois transporta o cabeçalho do protocolo de controle. O *sniffer* extrai com sucesso a informação de "ICMP echo request", permitindo identificar a natureza da operação original.
- **Processamento de Fragmentos Subsequentes (IDs 3 e 4):** Estes pacotes são identificados apenas como **IP**, com o summary "Datagrama Fragmentado". Tecnicamente, isto está correto e demonstra a precisão do processador desenvolvido: uma vez que o cabeçalho ICMP apenas existe no primeiro fragmento, os restantes contêm apenas *payload* de dados precedido pelo cabeçalho IP. O *sniffer* não "adivinha" o protocolo superior, reportando simplesmente o que deteta em cada pacote. No nosso código, incluímos uma verificação, aplicada pelas hosts que recebe, datagramas para verificar se um datagrama recebido é ou não um fragmento, sendo esta: **ip.frag != 0 or ip.flags.MF**, tal como se verifica no seguinte excerto de código;

```
"summary": "Datagrama Fragmentado" if (ip.frag != 0 or ip.flags.MF) else "Datagrama Normal"
```

Atividade de Rede

Permite monitorizar e analisar toda a atividade da rede em tempo real, desde o momento em que a captura é iniciada.

Q layer3.ip.flags==MF or layer3.ip.offset!=0

1-8 Proximo

Captura Parada

72 Resultados

>	2 ICMP	Fonte: 10.0.0.21	Destino: 10.0.0.10	Tamanho: 1514	Info: ICMP echo request	06/05/2026 15:34:47
>	3 IP	Fonte: 10.0.0.21	Destino: 10.0.0.10	Tamanho: 1514	Info: Datagrama Fragmentado	06/05/2026 15:34:47
>	4 IP	Fonte: 10.0.0.21	Destino: 10.0.0.10	Tamanho: 1082	Info: Datagrama Fragmentado	06/05/2026 15:34:47
>	5 ICMP	Fonte: 10.0.0.10	Destino: 10.0.0.21	Tamanho: 1514	Info: ICMP echo reply	06/05/2026 15:34:47
>	6 IP	Fonte: 10.0.0.10	Destino: 10.0.0.21	Tamanho: 1514	Info: Datagrama Fragmentado	06/05/2026 15:34:47
>	7 IP	Fonte: 10.0.0.10	Destino: 10.0.0.21	Tamanho: 1082	Info: Datagrama Fragmentado	06/05/2026 15:34:47
>	9 ICMP	Fonte: 10.0.0.21	Destino: 10.0.0.10	Tamanho: 1514	Info: ICMP echo request	06/05/2026 15:34:48
>	10 IP	Fonte: 10.0.0.21	Destino: 10.0.0.10	Tamanho: 1514	Info: Datagrama Fragmentado	06/05/2026 15:34:48
>	11 IP	Fonte: 10.0.0.21	Destino: 10.0.0.10	Tamanho: 1082	Info: Datagrama Fragmentado	06/05/2026 15:34:48
>	12 ICMP	Fonte: 10.0.0.10	Destino: 10.0.0.21	Tamanho: 1514	Info: ICMP echo reply	06/05/2026 15:34:48

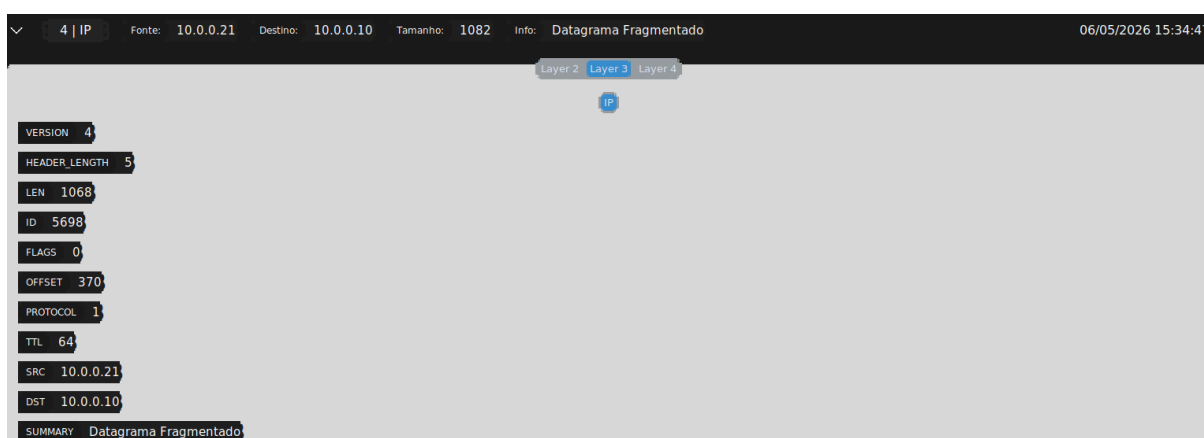
Foi desde logo possível, a partir do resumo apresentado para cada pacote, e também através da análise do cabeçalho IP de cada um, que todos os datagramas se tratam de fragmentos.

2 | ICMP Fonte: 10.0.0.21 Destino: 10.0.0.10 Tamanho: 1514 Info: ICMP echo request 06/05/2026 15:34:47

Layer 2 Layer 3 Layer 4

IP ICMP

VERSION	4
HEADER_LENGTH	5
LEN	1500
ID	5698
FLAGS	MF
OFFSET	0
PROTOCOL	1
TTL	64
SRC	10.0.0.21
DST	10.0.0.10
SUMMARY	Datagrama Fragmentado



Ao analisar os detalhes da camada de rede (Layer 3) para o mesmo datagrama original, confirmamos a correta dissecação dos campos de fragmentação:

- **Identificador Único (ID 5698):** Verifica-se que os três pacotes partilham o mesmo **ID 5698**. Isto é fundamental para que o nó de destino saiba que estes fragmentos pertencem todos ao mesmo pacote original de 4000 bytes, permitindo uma correta reconstituição do datagrama original
- **Gestão de Flags (MF - More Fragments):**
 - Nos pacotes de **ID 2** e **ID 3**, a flag **MF** está ativa, indicando que ainda existem mais fragmentos a caminho.
 - No pacote de **ID 4**, a flag está a **0**, sinalizando ao sistema que este é o último pedaço do datagrama, permitindo iniciar a remontagem.
- **Cálculo de Offset:**
 - O pacote de **ID 2** tem **Offset: 0** (início do pacote).
 - O pacote de **ID 3** tem **Offset: 185**.
 - O pacote de **ID 4** tem **Offset: 370**.
- *Nota técnica:* O Scapy mostra o offset em unidades de 8 bytes. Desta forma, o offset, como sendo o byte em concreto do payload do datagrama original, onde começa o payload daquele fragmento, seria em concreto:
 - ID 5: 0

- ID 6: $185 * 8 = 1480$
- ID 7: $370 * 8 = 2960$

Protocolo Superior: Em todos os fragmentos, o campo **protocol** mantém-se como **1** que é um valor que identifica o tipo do payload, ou seja o protocolo sobre o qual o payload do datagrama foi encapsulado, permitindo que o receptor saiba qual o protocolo a que os dados pertencem após a reconstrução.

Um dos pontos críticos validados por esta captura foi a gestão do **MTU (Maximum Transmission Unit)** e a decomposição dos tamanhos de pacote reportados pela aplicação. Embora o comando executado no PC1 tenha solicitado um *payload* de **4000 bytes (ping -s 4000)**, a análise dos fragmentos revela a estrutura real da pilha protocolar:

Cálculo do Datagrama Original:

Para que os 4000 bytes de dados úteis cheguem ao destino, o sistema adiciona os cabeçalhos das camadas superiores:

- **Payload de Dados:** 4000 bytes
- **Cabeçalho ICMP:** 8 bytes
- **Total a transportar na Camada de Rede (IP):** 4008 bytes

Distribuição pelos Fragmentos:

Dado que o MTU da interface eth0 é de **1500 bytes**, o protocolo IP segmentou o datagrama da seguinte forma, conforme visível nos detalhes do *sniffer*:

1. **Fragmento 1 (ID 2):** Apresenta um **LEN** de **1500** (20 bytes de cabeçalho IP + 1480 bytes de dados). O tamanho total da trama reportado é de **1514 bytes**, incluindo os 14 bytes do cabeçalho **Ethernet (Layer 2)**.
2. **Fragmento 2 (ID 3):** Segue a mesma lógica, transportando mais **1480 bytes** de dados (Total de 1514 na trama).
3. **Fragmento 3 (ID 4):** Transporta o remanescente dos dados. O **LEN** é de **1068** (20 bytes cabeçalho IP + 1048 bytes de dados). O tamanho total da trama é de **1082 bytes**.
4. Convém referir que o valor **5** que aparece no teu campo Header_Length refere-se ao número de palavras de **32 bits (4 bytes)** no cabeçalho IP.
 - Para calcular o tamanho em bytes do cabeçalho: $5 * 4 = 20$ **bytes**

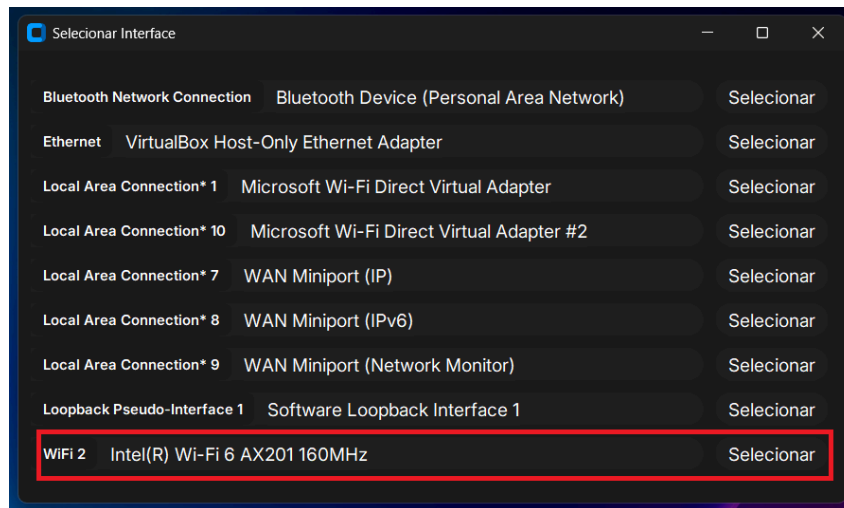
Verificação Matemática: A soma dos dados transportados ($1480 + 1480 + 1048$) totaliza exatamente os **4008 bytes** (4000 de *payload* + 8 de cabeçalho ICMP) necessários para a reconstrução do pacote original.

Parte B - Transposição para a interface real do PC

Esta fase consistiu em testar o *sniffer* fora do ambiente controlado do simulador CORE, executando-o na interface de rede real do computador. O objetivo principal foi validar a robustez do software perante o tráfego real de uma rede física.

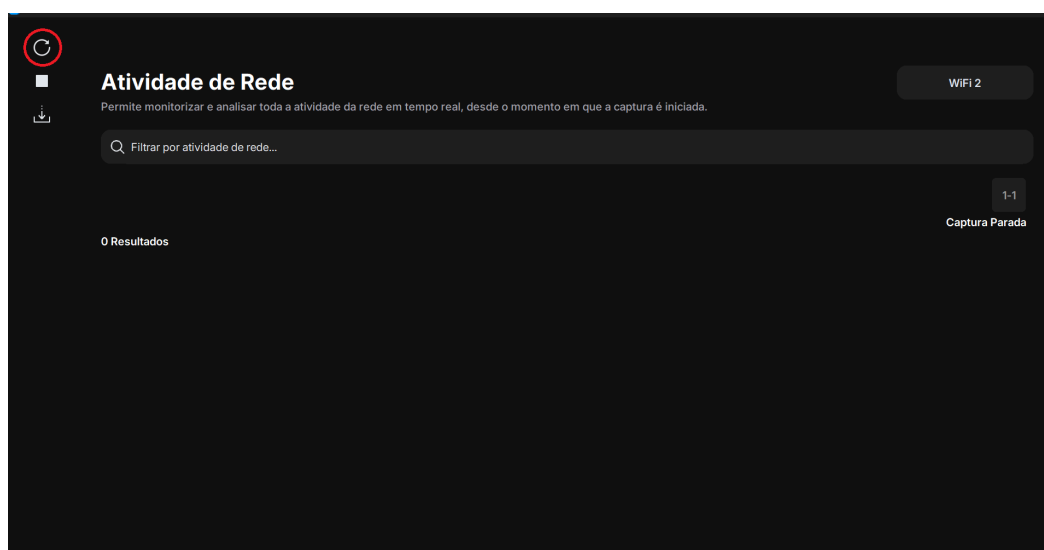
A transição permitiu confirmar que a lógica de processamento por camadas e o processo de filtragem mantêm a sua eficácia mesmo com o volume de tráfego superior. De modo a

explicar com coerência e clareza o processo de execução do sniffer na rede real do computador, vamos mostrar todos os passos provando assim o modo de funcionamento juntamente com determinados resultados específicos obtidos de modo a provar a atividade do nosso sniffer. Inicialmente escolhemos o tipo de interface que queremos selecionar para a captura do tráfego, neste caso iremos escolher a interface **WIFI 2** como se apresenta na figura seguinte rodeada.



Após a seleção da interface, somos encaminhados para o **Dashboard principal**, é aqui que é realizada a gestão de todo o tráfego capturado segundo a interface escolhida anteriormente. Nesta interface, é possível efetuar a filtragem, alternar entre interfaces, gerir o estado da captura e por fim exportar os dados obtidos.

Conforme ilustrado na figura seguinte, o sistema inicia com a **"Captura Parada"**. Para dar início à recolha de dados, utiliza-se o botão de rotação (assinalado com o círculo vermelho). Uma vez em execução, a captura pode ser interrompida a qualquer momento através do botão com formato quadrado logo abaixo. Por fim, o ícone de exportação, situado logo a seguir ao botão de parar a captura, permite guardar todos os dados obtidos para análise posterior em formato JSON.



Após a demonstração da interface inicial, procedemos ao início da captura de tráfego na rede real. Para validar a robustez e a precisão do *sniffer* de forma controlada, optámos por gerar tráfego específico no sistema através da execução do comando “**tracert google.com**” no terminal, juntamente com o browser aberto.

O propósito de utilizar o comando **tracert** foi forçar a captura de uma sequência de pacotes ICMP/UDP, simultaneamente, a utilização do browser serviu o propósito de gerar tráfego em (HTTP/HTTPS) e pacotes TCP. Na figura seguinte está apresentado o comando “**tracert google.com**”.

```
PS C:\Users\tiago> tracert google.com

Tracing route to google.com [216.58.198.46]
over a maximum of 30 hops:

  1    6 ms    1 ms    1 ms  172.26.254.254
  2    9 ms    2 ms    2 ms  172.16.10.3
  3    1 ms    2 ms    1 ms  endpub001.firewall-eduroam.uminho.pt [193.137.92.1]
  4    7 ms    1 ms    1 ms  172.16.4.3
  5    4 ms    1 ms    1 ms  Router41.Porto.fccn.pt [193.136.4.101]
  6    8 ms    2 ms    2 ms  194.210.7.216
  7   17 ms    5 ms    5 ms  Router2.Porto.fccn.pt [194.210.7.203]
  8   12 ms    3 ms    4 ms  fccn-ias-geant-gw.rtl.por.pt.geant.net [83.97.88.44]
  9   37 ms   37 ms   37 ms  google.rtl.ams.nl.geant.net [83.97.89.46]
 10   43 ms   53 ms   38 ms  google-gw.rtl.ams.nl.geant.net [83.97.89.47]
 11   62 ms   38 ms   47 ms  142.251.70.73
 12   44 ms   39 ms   39 ms  108.170.227.9
 13   44 ms   43 ms   38 ms  mil04s04-in-f14.1e100.net [216.58.198.46]

Trace complete.
PS C:\Users\tiago> |
```

O propósito destas ações é verificar se o nosso *sniffer* consegue responder a tudo da forma mais clara, garantindo que o sistema mantém a **fidelidade na captura** mesmo estando na interface da rede real. Com isto, pretendemos validar se o programa consegue capturar corretamente cada atividade: confirmando que a execução do **tracert** resulta na captura de pacotes ICMP e que a navegação web é devidamente identificada através de tráfego TCP. Desta forma, provamos que o *sniffer* não se limita a captar dados aleatórios, mas sim que correlaciona com precisão o tráfego gerado com os protocolos correspondentes, assegurando que captura corretamente o tráfego observado.

Após termos recolhido uma boa quantidade de tráfego, optámos por interromper a captura antes de aplicarmos os filtros. Embora o nosso *sniffer* permita a filtragem dinâmica em tempo real, ou seja, enquanto estamos a capturar podemos colocar o filtro e a captura é feita segundo esse mesmo filtro, decidimos realizar esta análise com a captura parada para não estar a capturar tráfego por tempo indefinido portanto a imagem seguinte apresenta o nosso caso de estudos e teste na rede real.

Atividade de Rede

WIFI 2

Permite monitorizar e analisar toda a atividade da rede em tempo real, desde o momento em que a captura é iniciada.

Q |

1-123

Proximo

Captura Parada

1227 Resultados

>	1 HTTPS	Fonte: 172.26.127.133	Destino: 13.107.137.11	Tamanho: 2741	Info: TCP (HTTPS)	23/04/2026 19:05:36
>	2 HTTPS	Fonte: 172.26.127.133	Destino: 13.107.137.11	Tamanho: 1099	Info: TCP (HTTPS)	23/04/2026 19:05:36
>	3 HTTPS	Fonte: 13.107.137.11	Destino: 172.26.127.133	Tamanho: 54	Info: TCP ACK (HTTPS)	23/04/2026 19:05:36
>	4 HTTPS	Fonte: 13.107.137.11	Destino: 172.26.127.133	Tamanho: 54	Info: TCP ACK (HTTPS)	23/04/2026 19:05:36
>	5 HTTPS	Fonte: 13.107.137.11	Destino: 172.26.127.133	Tamanho: 2554	Info: TCP ACK (HTTPS)	23/04/2026 19:05:37
>	6 HTTPS	Fonte: 13.107.137.11	Destino: 172.26.127.133	Tamanho: 531	Info: TCP (HTTPS)	23/04/2026 19:05:37
>	7 HTTPS	Fonte: 172.26.127.133	Destino: 13.107.137.11	Tamanho: 54	Info: TCP ACK (HTTPS)	23/04/2026 19:05:37

Testes à captura/ Aplicação de Filtros

Filtro ICMP

No primeiro exemplo de filtragem na rede real, utilizamos o filtro “**protocol == ICMP**”. O objetivo deste teste foi demonstrar que o nosso sniffer responde de forma correta à utilização de uma determinada condição e ver como o protocolo ICMP se comporta.

Como se pode observar na figura seguinte, o *sniffer* filtrou com sucesso os pacotes com protocolo **ICMP** resultantes do comando tracert. Conseguimos perceber de forma fácil e intuitiva que o filtro foi aplicado ao ver inicialmente que o número de resultados agora é bastante inferior já que conseguimos obter 78 resultados de 1227. Conseguimos ainda observar a informação mais relevante relativa a cada pacote, a sua fonte, destino, tamanho do pacote e o tipo de mensagem ICMP (ICMP echo request, Time exceeded, ICMP echo request).

Atividade de Rede

WIFI 2

Permite monitorizar e analisar toda a atividade da rede em tempo real, desde o momento em que a captura é iniciada.

Q protocol==ICMP

1-8

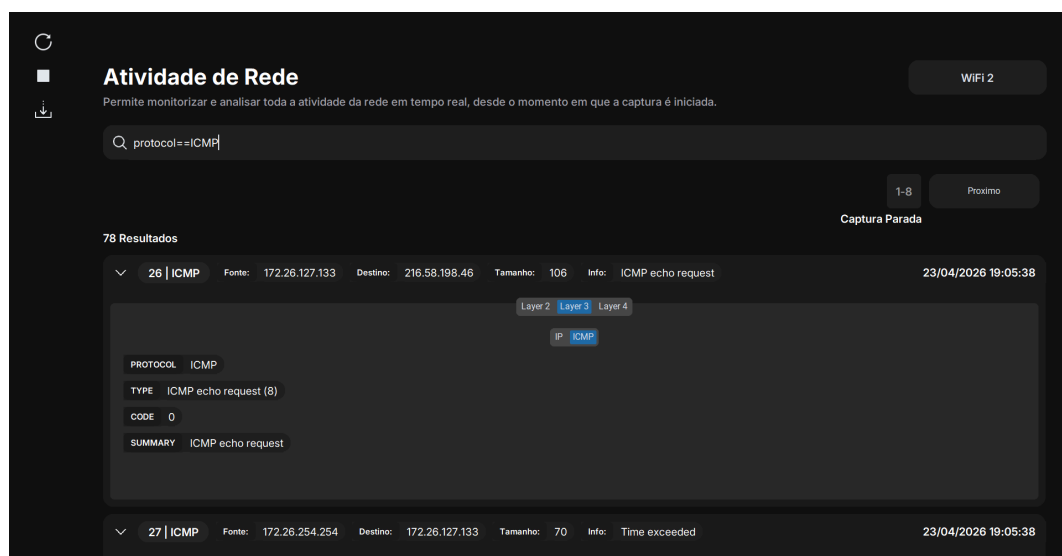
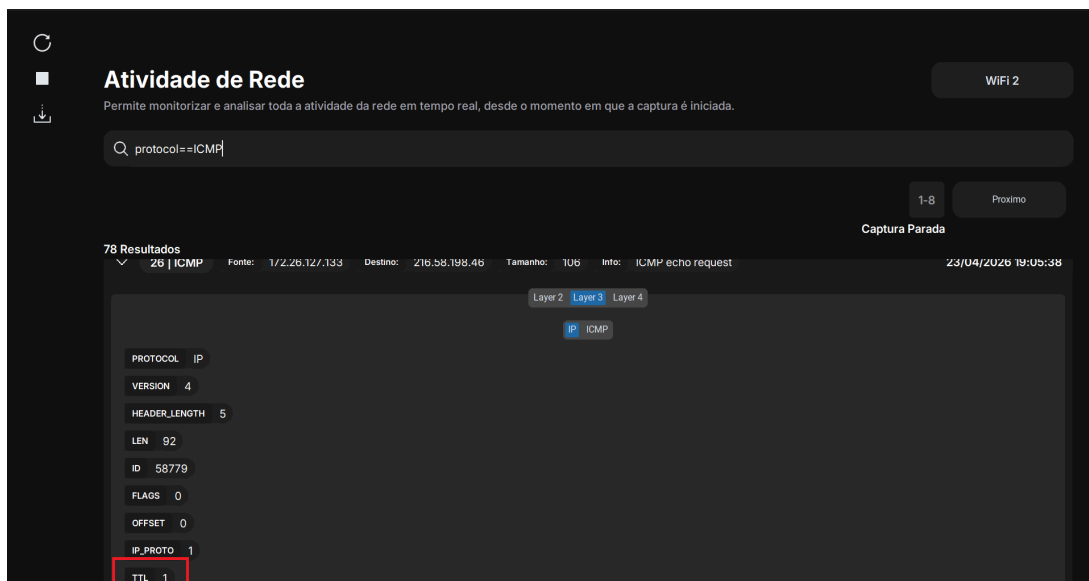
Proximo

Captura Parada

78 Resultados

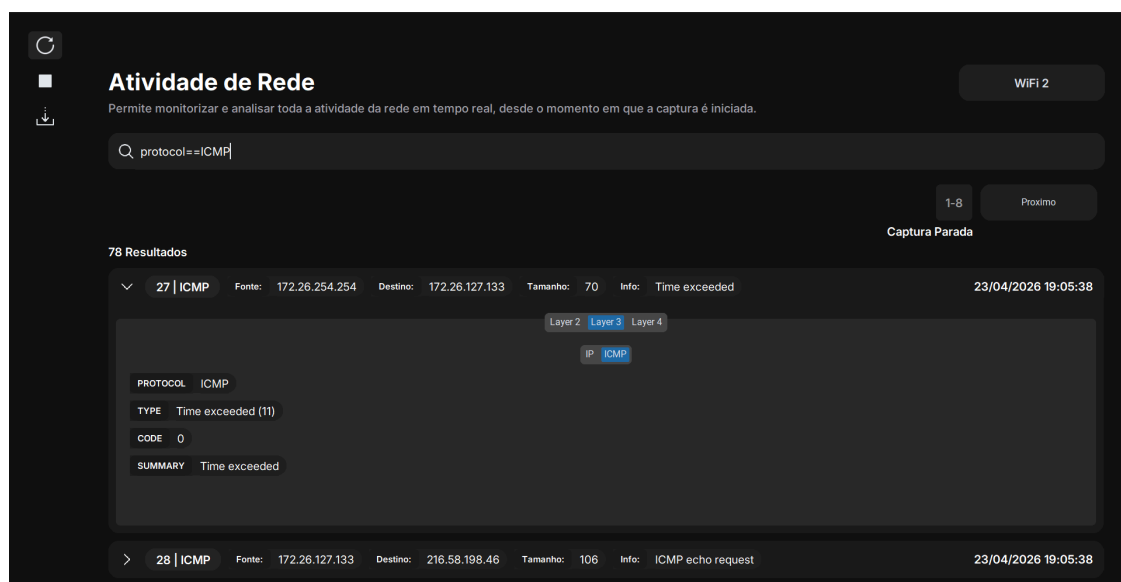
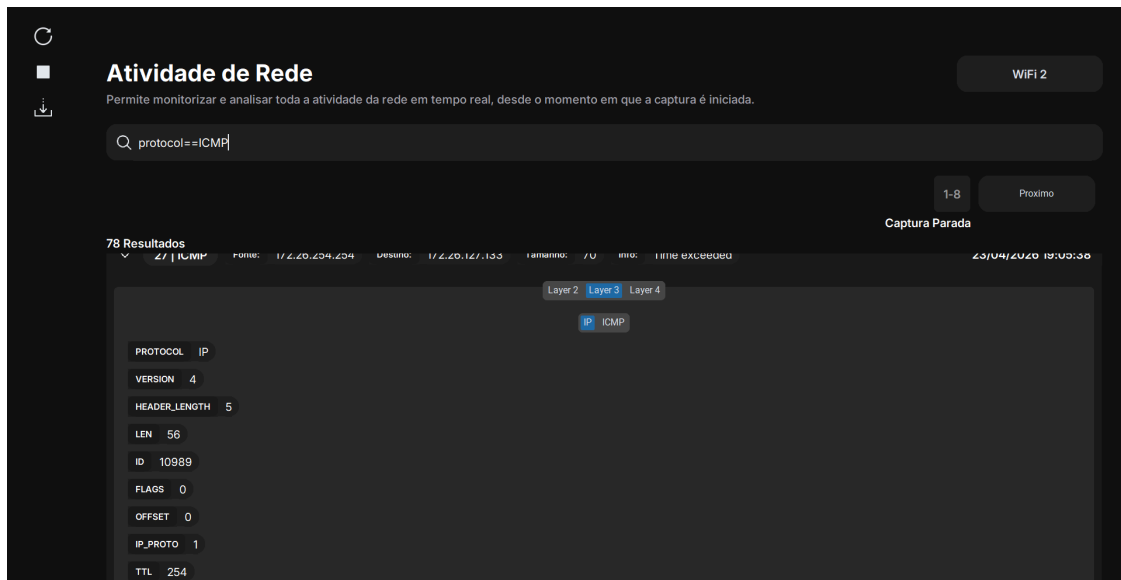
>	26 ICMP	Fonte: 172.26.127.133	Destino: 216.58.198.46	Tamanho: 106	Info: ICMP echo request	23/04/2026 19:05:38
>	27 ICMP	Fonte: 172.26.254.254	Destino: 172.26.127.133	Tamanho: 70	Info: Time exceeded	23/04/2026 19:05:38
>	28 ICMP	Fonte: 172.26.127.133	Destino: 216.58.198.46	Tamanho: 106	Info: ICMP echo request	23/04/2026 19:05:38
>	29 ICMP	Fonte: 172.26.254.254	Destino: 172.26.127.133	Tamanho: 70	Info: Time exceeded	23/04/2026 19:05:38
>	30 ICMP	Fonte: 172.26.127.133	Destino: 216.58.198.46	Tamanho: 106	Info: ICMP echo request	23/04/2026 19:05:38
>	31 ICMP	Fonte: 172.26.254.254	Destino: 172.26.127.133	Tamanho: 70	Info: Time exceeded	23/04/2026 19:05:38
>	198 ICMP	Fonte: 172.26.127.133	Destino: 216.58.198.46	Tamanho: 106	Info: ICMP echo request	23/04/2026 19:05:44

Como podemos verificar, conseguimos capturar mensagens **ICMP Echo Request (ICMP Type 8)**, já que estas são usadas para testar a ligação entre o nosso host e o destino, verificando se o equipamento está acessível na rede. Podemos observar que o nosso sniffer está totalmente sincronizado com o que o terminal apresenta, uma vez que o endereço de origem da mensagem é o host com **IP 172.26.127.133** e o endereço de destino é **216.58.198.46**. Assim, confirma-se que a mensagem foi enviada com sucesso. Ao analisar o **pacote 26** com o protocolo ICMP, obtemos um **TTL = 1**, o que indica que o pacote ainda possui tempo de vida suficiente para alcançar o destino. Como podemos ver na figura abaixo, toda esta informação pode ser claramente observada.



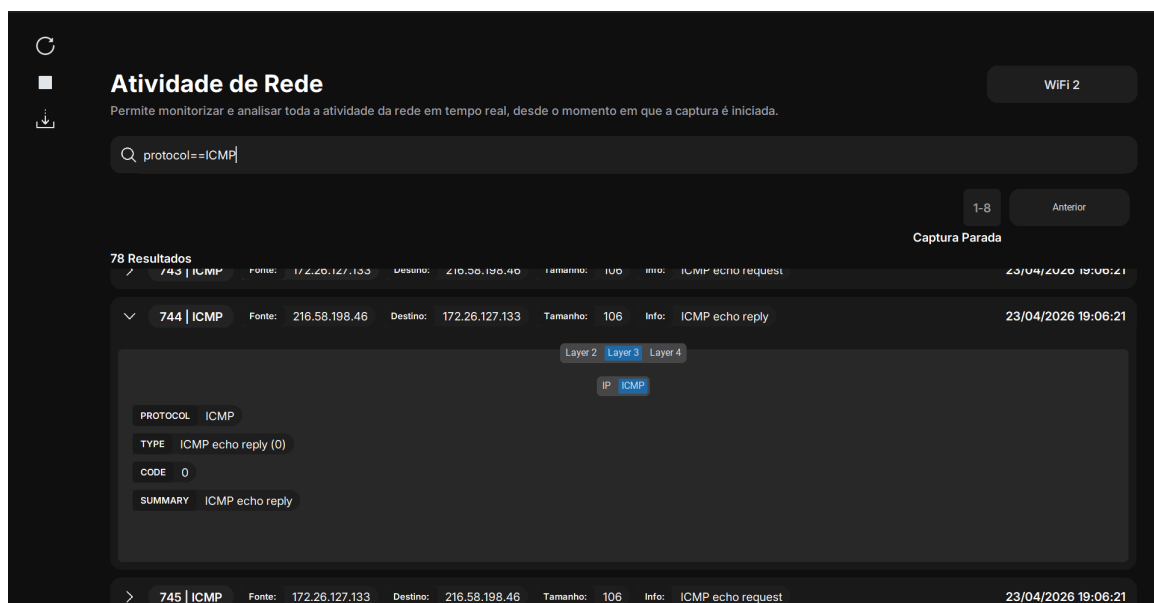
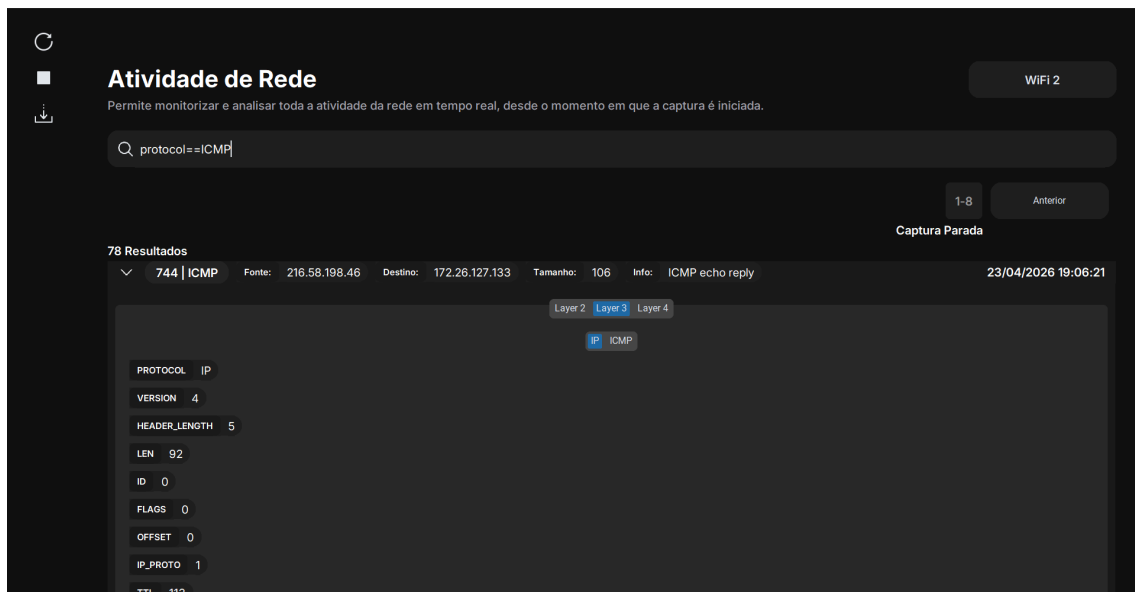
Por sua vez, os pacotes ICMP com a mensagem **Time Exceeded** (Type 11) capturados indicam que o sistema está a monitorizar corretamente as respostas dos routers. Esta mensagem é gerada quando um pacote atinge TTL = 0, sendo então descartado e enviada uma notificação de erro ao remetente. Pela figura seguinte, verifica-se que o TTL = 254, confirmando que a mensagem de erro foi corretamente gerada.

Como podemos observar, o router associado ao primeiro salto (**IP 172.26.254.254**) detetou o esgotamento do TTL do pacote e enviou a mensagem de erro para o endereço IP do host (**172.26.127.133**). A captura bem-sucedida confirma que a aplicação consegue interpretar corretamente mensagens de erro de rede, identificando os routers envolvidos no caminho até ao destino. Nas figuras abaixo, observa-se também o valor do TTL e os restantes parâmetros do protocolo IP, bem como a identificação correta do tipo de mensagem ICMP, respetivamente.



Por fim, conseguimos capturar pacotes **ICMP Echo Reply (Type 0)**, que indicam que o destino recebeu o pedido e respondeu com sucesso ao host. Como podemos observar pelos endereços IP na figura, o remetente desta mensagem é o **216.58.198.46**, o que

comprova que o servidor da Google respondeu diretamente ao endereço IP do nosso host (**172.26.127.133**). A receção destes pacotes no nosso *sniffer* valida que a rota de rede está totalmente funcional, confirmando que os dados atravessaram com sucesso todos os routers até alcançarem o destino. O facto do destino ter conseguido responder confirma que os pacotes chegaram devidamente e que o TTL definido foi suficiente para que a mensagem chegasse com sucesso. Esta captura encerra o ciclo do protocolo ICMP, demonstrando a capacidade do nosso sniffer em conseguir capturar todas as mensagens do protocolo ICMP, provando assim que está devidamente funcional.

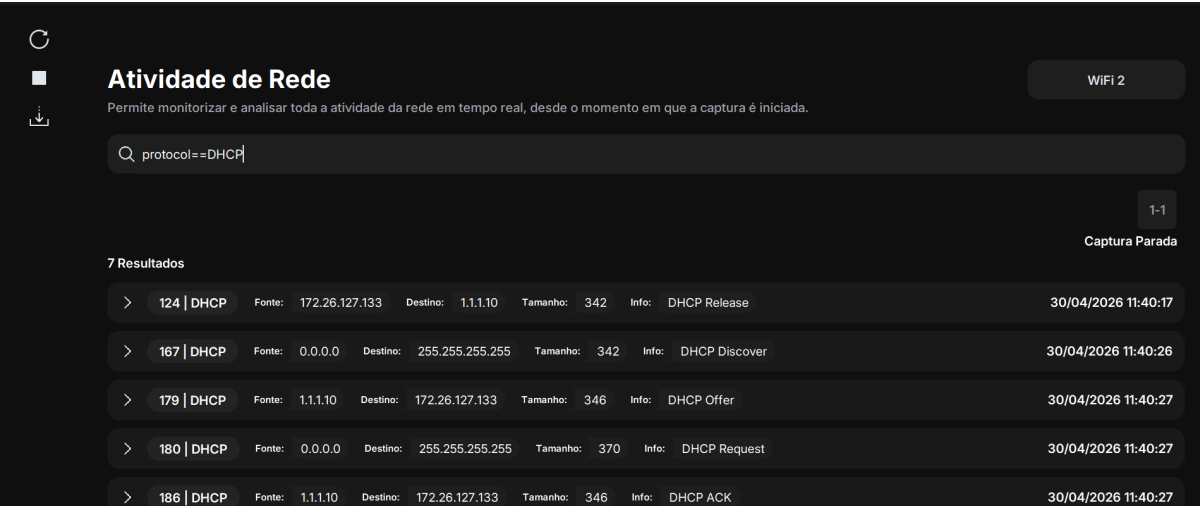


Filtro DHCP

Para capturar o tráfego DHCP, foi iniciada a captura no sniffer. De seguida, no terminal foram executados os comandos **ipconfig /release** e **ipconfig /renew**, com o objetivo de forçar a libertação do endereço IP atual e solicitar uma nova configuração ao servidor DHCP. Este procedimento permitiu desencadear o processo completo de atribuição do endereço IP como conseguimos verificar na figura seguinte pela aplicação do filtro “**protocol == DHCP**”.

Durante a captura, foi possível observar a presença de um pacote **DHCP Release** (DHCP 7), que ocorre devido ao comando **/release**, indicando que o cliente está a libertar o endereço IP anteriormente atribuído. Este comportamento é normal e faz parte do processo de renovação da configuração de rede, para que o DHCP consiga assim atribuir um novo endereço IP ao host na determinada rede.

Posteriormente, foram capturadas com sucesso todas as mensagens do processo **DORA** relativos ao pacote DHCP (Discover, Offer, Request e Acknowledge). O **DHCP Discover** corresponde ao pedido inicial do cliente para obtenção de um endereço IP, o **DHCP Offer** é a resposta do servidor com uma proposta de endereço IP, o **DHCP Request** indica a aceitação dessa oferta por parte do cliente e o **DHCP Acknowledge** confirma a atribuição final do endereço IP. A captura destas mensagens demonstra que o nosso sniffer conseguiu registar corretamente todo o processo DHCP, validando a comunicação entre o cliente e o servidor confirmando o funcionamento adequado da atribuição de endereços IP. Isto evidencia que o processo decorre de forma correta e consistente, demonstrando o comportamento esperado do protocolo na rede e a correta captura por parte do nosso sniffer evidenciando todos os passos esperados. Na figura seguinte conseguimos observar o filtro aplicado juntamente com o comportamento esperado por parte do protocolo DHCP.



The screenshot shows the 'Atividade de Rede' (Network Activity) window in Wireshark. The filter bar at the top contains the text 'protocol==DHCP'. Below the filter bar, it indicates '7 Resultados' (7 Results). The table lists the following packets:

Packet #	Protocol	Source (Fonte)	Destination (Destino)	Size (Tamanho)	Info	Time
124	DHCP	172.26.127.133	1.1.1.10	342	DHCP Release	30/04/2026 11:40:17
167	DHCP	0.0.0.0	255.255.255.255	342	DHCP Discover	30/04/2026 11:40:26
179	DHCP	1.1.1.10	172.26.127.133	346	DHCP Offer	30/04/2026 11:40:27
180	DHCP	0.0.0.0	255.255.255.255	370	DHCP Request	30/04/2026 11:40:27
186	DHCP	1.1.1.10	172.26.127.133	346	DHCP ACK	30/04/2026 11:40:27

Filtro HTTP

No terceiro teste de filtragem na rede real, utilizámos o filtro “**protocol == HTTP and length >= 60**”. O objetivo foi isolar o tráfego de navegação web, focando em pacotes com um

tamanho mínimo de 60 bytes, possibilitando uma análise mais clara das comunicações entre o browser e o servidor.

Como se pode observar na captura, o sniffer identificou com sucesso uma sequência de pacotes **TCP SYN** (associados ao estabelecimento da ligação HTTP) partindo do host (172.26.244.235) em direção ao servidor (34.223.124.45). Estes pacotes representam a fase inicial do estabelecimento da conexão, onde o cliente solicita o início de uma sessão de comunicação antes da troca de dados.

Posteriormente, foram observados pacotes **TCP ACK**, que indicam a confirmação da ligação por parte do servidor, estabelecendo assim o canal de comunicação entre as duas entidades. A presença destes pacotes, demonstra que o sniffer consegue monitorizar não apenas a transmissão de dados, mas também o estabelecimento e gestão das sessões que suportam o protocolo HTTP. Esta análise valida a eficácia do filtro aplicado, comprovando que a ferramenta consegue identificar corretamente o encapsulamento de HTTP sobre TCP e acompanhar o estabelecimento de uma sessão de comunicação web em tempo real. Comprovando que o sniffer consegue capturar tráfego de rede de forma correta e consistente.

Atividade de Rede

WiFi 2

Permite monitorizar e analisar toda a atividade da rede em tempo real, desde o momento em que a captura é iniciada.

Q protocol==HTTP and length>=60

1-2Proximo

Captura Parada

16 Resultados

> 329 | HTTP

Fonte: 172.26.244.235

Destino: 34.223.124.45

Tamanho: 66

Info: TCP SYN (HTTP)

23/04/2026 21:32:46

> 330 | HTTP

Fonte: 172.26.244.235

Destino: 34.223.124.45

Tamanho: 66

Info: TCP SYN (HTTP)

23/04/2026 21:32:46

> 339 | HTTP

Fonte: 172.26.244.235

Destino: 34.223.124.45

Tamanho: 66

Info: TCP SYN (HTTP)

23/04/2026 21:32:47

> 340 | HTTP

Fonte: 172.26.244.235

Destino: 34.223.124.45

Tamanho: 66

Info: TCP SYN (HTTP)

23/04/2026 21:32:47

> 351 | HTTP

Fonte: 172.26.244.235

Destino: 34.223.124.45

Tamanho: 66

Info: TCP SYN (HTTP)

23/04/2026 21:32:47

> 354 | HTTP

Fonte: 172.26.244.235

Destino: 34.223.124.45

Tamanho: 502

Info: TCP (HTTP)

23/04/2026 21:32:47

> 366 | HTTP

Fonte: 34.223.124.45

Destino: 172.26.244.235

Tamanho: 590

Info: TCP ACK (HTTP)

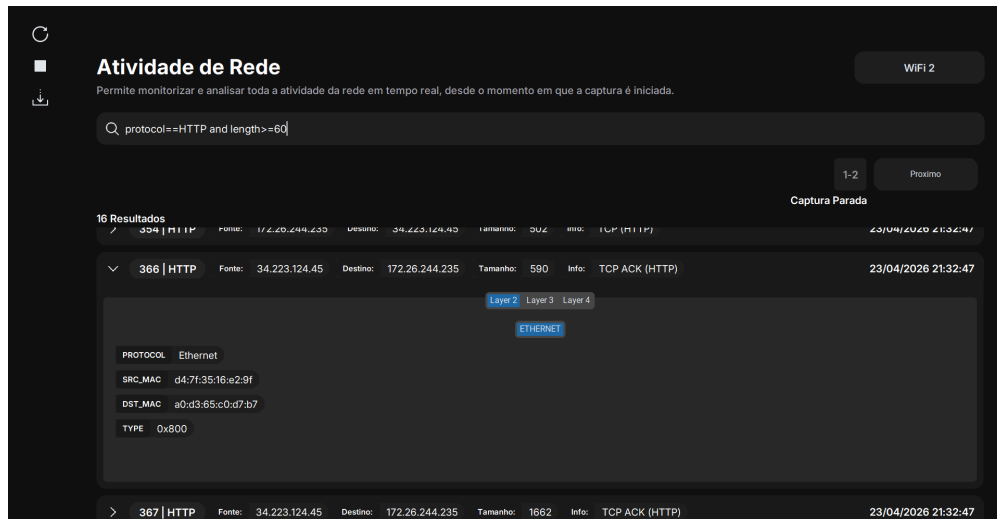
23/04/2026 21:32:47

De modo a validar a profundidade de análise do nosso sniffer, selecionamos o pacote 366 (com um tamanho de 590 bytes) de modo a estudar todos os parâmetros que este apresenta. Este pacote é relevante pois representa uma resposta vinda do servidor web para o nosso host. Abaixo, detalhamos a informação extraída pelo *sniffer* em cada camada:

1. Layer 2 – Ethernet (Link Layer)

Nesta camada, o sniffer identifica os endereços físicos (MAC) das interfaces envolvidas:

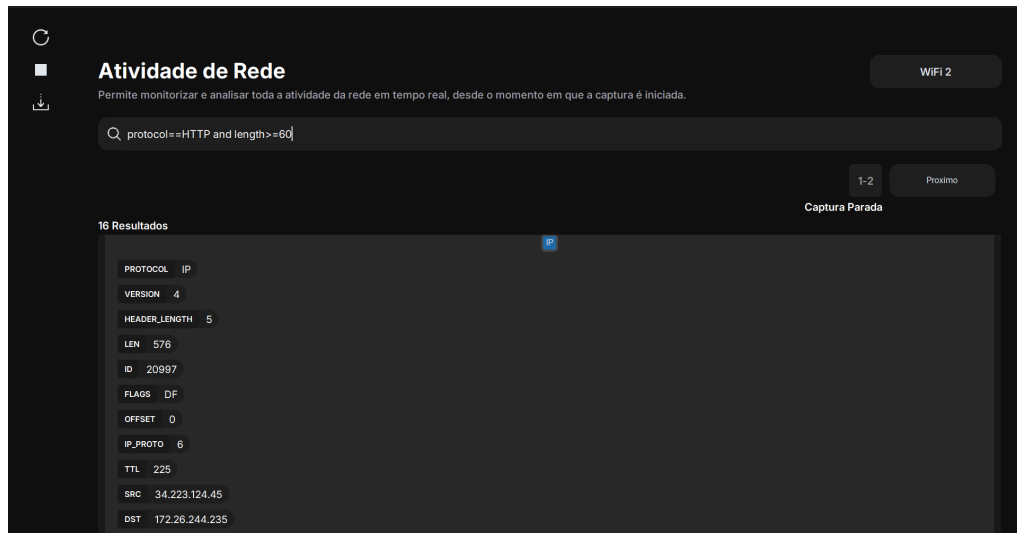
- **SRC_MAC (d4:7f:35:16:e2:9f)**: corresponde ao endereço MAC do gateway (router) que encaminha o tráfego para a rede local.
- **DST_MAC (a0:d3:65:c0:d7:b7)**: corresponde ao endereço MAC da nossa placa de rede.
- **TYPE (0x0800)**: indica que o protocolo encapsulado no frame Ethernet é IPv4.



2. Layer 3 – IP (Network Layer)

Aqui observamos a lógica de endereçamento e o comportamento do pacote na rede:

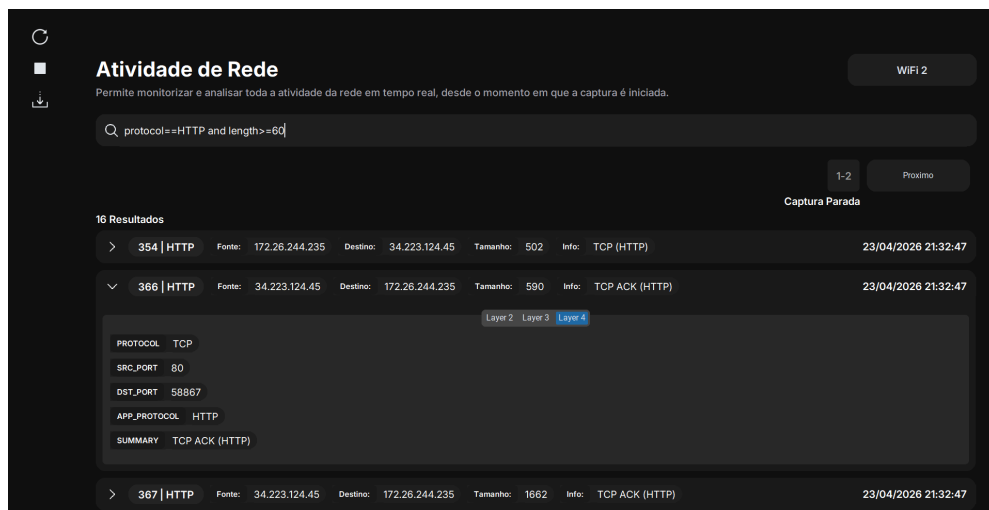
- **Protocol (IP)**: indica o protocolo de transporte utilizado; o valor 6 identifica que o conteúdo encapsulado é TCP.
- **Version (4)**: confirma que a comunicação utiliza o protocolo IPv4.
- **Header Length (5)**: indica o tamanho do cabeçalho IP, correspondendo a 20 bytes.
- **Len (576)**: representa o tamanho total do pacote nesta camada, sendo 20 bytes de cabeçalho e 556 bytes de dados.
- **ID (20997)**: identificador único do pacote.
- **FLAGS (DF)**: a flag *Don't Fragment* indica que o pacote não pode ser fragmentado durante o seu percurso.
- **Offset (0)**: indica que o pacote não é fragmentado e representa a unidade original.
- **IP_PROTO (6)**: identifica o protocolo de transporte utilizado, neste caso TCP.
- **TTL (225)**: valor de Time To Live que indica o tempo de vida restante do pacote na rede.
- **SRC (34.223.124.45)**: endereço IP de origem, correspondente ao servidor remoto.
- **DST (172.26.244.235)**: endereço IP de destino, correspondente ao nosso host.



3. Layer 4 – TCP (Transport Layer)

Nesta camada, o sniffer identifica a comunicação da seguinte forma:

- **SRC_PORT (80):** indica que o servidor utiliza a porta 80, associada ao protocolo HTTP.
- **DST_PORT (58867):** corresponde a uma porta utilizada pelo browser para receber a resposta.
- **APP_PROTOCOL (HTTP):** confirma que o tráfego pertence à camada de aplicação HTTP, apesar de ser transportado por TCP.
- **SUMMARY (TCP ACK):** indica que o pacote confirma a receção de dados enviados anteriormente pelo cliente.



Limitações da Aplicação Desenvolvida

Uma das limitações da aplicação está relacionada com a ausência de diagramas sobre o funcionamento dos protocolos. Esta foi uma opção tomada durante o desenvolvimento, uma vez que se considerou mais relevante concentrar a informação diretamente na interface da aplicação. Em vez de apresentar um diagrama separado para cada protocolo, a aplicação organiza os dados capturados por camadas, disponibilizando os campos principais de cada protocolo analisado. Desta forma, é possível observar diretamente na captura as informações associadas aos protocolos permitindo ao utilizador interpretar as trocas características de cada um através dos próprios dados recolhidos.

Assim, a inclusão de diagramas foi considerada menos necessária, pois a aplicação já apresenta a informação essencial de forma estruturada e integrada na interface. No entanto, esta decisão pode ser vista como uma limitação, uma vez que os diagramas poderiam facilitar a compreensão visual do funcionamento de alguns protocolos.

Outra limitação identificada está relacionada com a apresentação dos pacotes capturados. A aplicação não utiliza um scroll contínuo para mostrar todo o histórico, optando antes por dividir os pacotes em páginas. Esta decisão foi tomada porque o uso de um scroll prolongado causava erros e problemas de desempenho quando o número de pacotes aumentava. Desta forma, a paginação foi adotada como uma alternativa mais estável e organizada, permitindo consultar os pacotes de forma controlada e evitando sobrecarregar a interface. Apesar disso, pode ser considerada uma limitação, pois obriga o utilizador a navegar entre páginas em vez de visualizar toda a captura de forma contínua.

Conclusão do Trabalho Efetuado

Com a realização deste trabalho prático, foi possível desenvolver uma aplicação capaz de capturar, processar e apresentar tráfego de rede de forma organizada e compreensível. Através da utilização da biblioteca Scapy, a aplicação consegue analisar diferentes protocolos, como Ethernet, ARP, IP, ICMP, TCP, UDP e DHCP, extraíndo toda a informação relevante de cada pacote capturado.

A organização dos dados por camadas permitiu representar a estrutura dos pacotes de forma clara, facilitando a identificação dos protocolos envolvidos e das trocas características observadas na rede. Além disso, a implementação de filtros, paginação, histórico de pacotes e mecanismos de persistência em JSON tornou a aplicação mais completa, permitindo ao utilizador consultar, selecionar e guardar a informação capturada para análise posterior.

Em suma, este trabalho permitiu consolidar conhecimentos sobre protocolos de rede, funcionamento das camadas de comunicação e análise de pacotes, ao mesmo tempo que proporcionou experiência prática no desenvolvimento de uma ferramenta funcional de monitorização e estudo de tráfego de rede.